

The background features a dark blue gradient with faint, light blue concentric circles and degree markings (40, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260) on the left side. Several circular arrows indicate a clockwise direction of rotation.

JOI 2021/2022 本選 1 インターカステラー (INTERCASTELLAR)

解説: 平木 康傑



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

14

9

8

12



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

14

9

8

6

6



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

14	9	8	6	3	3
----	---	---	---	---	---



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

14	9	8	3	3	3	3
----	---	---	---	---	---	---



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

7	7	9	1	1	1	1	1	1	1	1	3	3	3	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



問題概要

- カステラが N ピース並んであり、左から i 個目の長さは A_i
- 偶数のピースを 2 等分する操作を繰り返す
- 操作終了後、左から X_j 個目の長さは？

7	7	9	1	1	1	1	1	1	1	1	3	3	3	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



ラテ

考察

- あるピースの分割が他のピースに影響することはない
- なので、各ピースについて別々に分割結果を考えて、それをつなげればよい

7	7	9	1	1	1	1	1	1	1	1	3	3	3	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

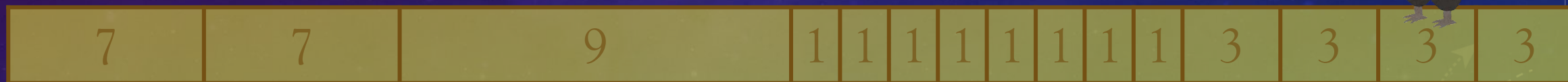
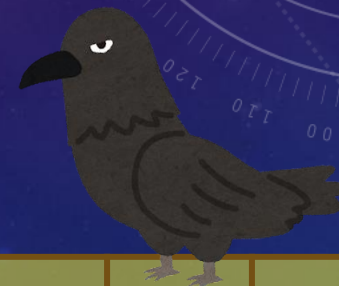
???



考察 (この時点で小課題 1 は解ける)

- あるピースの分割が他のピースに影響することはない
- なので、各ピースについて別々に分割結果を考えて、それをつなげればよい

???



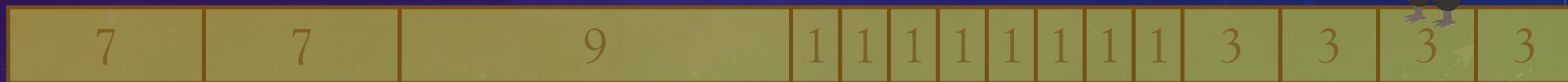
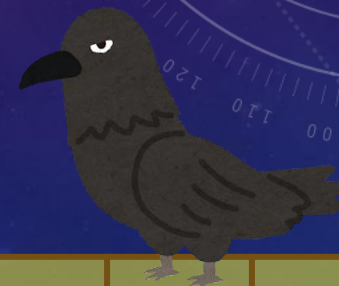
- 小課題 1 ($A_i \leq 8$) は長さに応じた 8 通りの分割を列挙できるので、各ピースの分割結果をつなげることで全体の分割結果がそのまま得られる
- 質問にはそのまま答える
- 25 点 GET



考察 (この時点で小課題 1 は解ける)

- あるピースの分割が他のピースに影響することはない
- なので、各ピースについて別々に分割結果を考えて、それをつなげればよい

カラス



- 小課題 1 ($A_i \leq 8$) は長さに応じた 8 通りの分割を列挙できるので、各ピースの分割結果をつなげることで全体の分割結果がそのまま得られる
- 質問にはそのまま答える
- 25 点 GET



ピースの分割結果を見て気付くこと

- 長さ $2^a \cdot b$ (b は奇数) のピースを分割しきると、長さ b のピース 2^a 個になる
 - ↑ この数式の意味はこう: 長さを 2 で割れるだけ割ったとき、 a は何回割れるか、 b は割った後の商
 - たとえば、長さ $120 (= 2^3 \cdot 15)$ のピースを分割しきると長さ 15 のピース $8 (= 2^3)$ 個になる

???
微か

$2^3 \cdot \text{奇数}$

$2^2 \cdot \text{奇数}$

$2^2 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

奇数

奇数

奇数

奇数

奇数

奇数

奇数

奇数

ピースの分割結果を見て気付くこと

- 長さ $2^a \cdot b$ (b は奇数) のピースを分割しきると、長さ b のピース 2^a 個になる
 - ↑ この数式の意味はこう: 長さを 2 で割れるだけ割ったとき、 a は何回割れるか、 b は割った後の商
 - たとえば、長さ $120 (= 2^3 \cdot 15)$ のピースを分割しきると長さ 15 のピース $8 (= 2^3)$ 個になる

かすか
微か

$2^3 \cdot \text{奇数}$

$2^2 \cdot \text{奇数}$

$2^2 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

$2^1 \cdot \text{奇数}$

奇数

奇数

奇数

奇数

奇数

奇数

奇数

奇数

分割結果の構造を利用して

- 長さ $2^a \cdot b$ (b は奇数) のピースを分割しきると、長さ b のピース 2^a 個になる
- ピースが何個に分割されるかが分かるので、これを利用したい
- 質問で聞かれた位置に来るピースが、 N 個のピースのうちどれに由来するものかがわかれば、そのピースの情報から分割後の長さが分かる

7 が 2 個

9 が 1 個

1 が 8 個

3 が 4 個

「左から 5 個目」はここに入るから、1 が答えだな

??????



分割結果の構造を利用して (これで小課題 2 も解ける)

- 長さ $2^a \cdot b$ (b は奇数) のピースを分割しきると、長さ b のピース 2^a 個になる
- ピースが何個に分割されるかが分かるので、これを利用したい
- 質問で聞かれた位置に来るピースが、 N 個のピースのうちどれに由来するものかがわかれば、そのピースの情報から分割後の長さが分かる
- 素直に実装すると: 左から個数を足していって、はじめに X_j を超えたところが求めるピース
 - 各質問に高々 N ステップで答えられるので、小課題 2 ($N \leq 1000, Q \leq 1000$) なら間に合う → 35 点 GET

7 が 2 個

累計 2 個.
足りない...

9 が 1 個

累計 3 個.
足りない...

1 が 8 個

累計 11 個.
はじめて 5 以上になった!

3 が 4 個

「左から 5 個目」はここに入るから、1 が答えだな

??????



分割結果の構造を利用して (これで小課題 2 も解ける)

- 長さ $2^a \cdot b$ (b は奇数) のピースを分割しきると、長さ b のピース 2^a 個になる
- ピースが何個に分割されるかが分かるので、これを利用したい
- 質問で聞かれた位置に来るピースが、 N 個のピースのうちどれに由来するものかがわかれば、そのピースの情報から分割後の長さが分かる
- 素直に実装すると: 左から個数を足していった、はじめに X_j を超えたところが求めるピース
 - 各質問に高々 N ステップで答えられるので、小課題 2 ($N \leq 1000, Q \leq 1000$) なら間に合う → 35 点 GET

7 が 2 個

累計 2 個.
足りない...

9 が 1 個

累計 3 個.
足りない...

1 が 8 個

累計 11 個.
はじめて 5 以上になった!

3 が 4 個

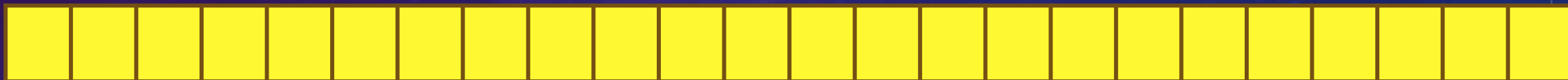
「左から 5 個目」はここに入るから、1 が答えだな



カラカラ帝

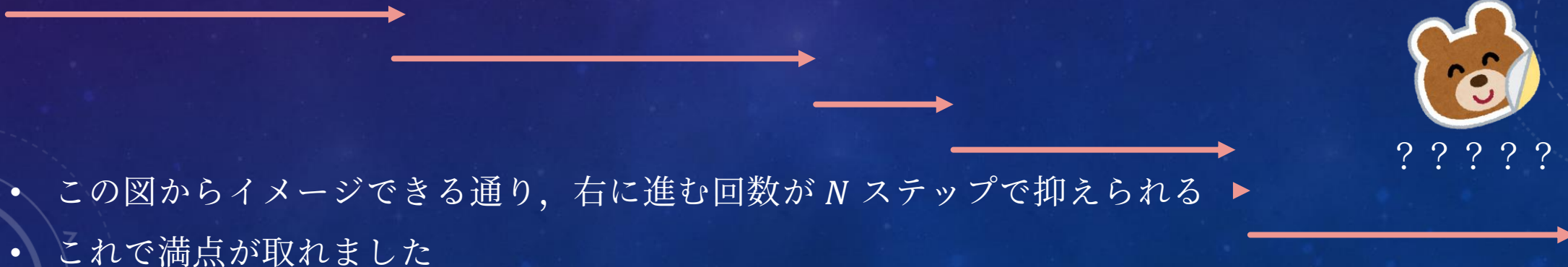
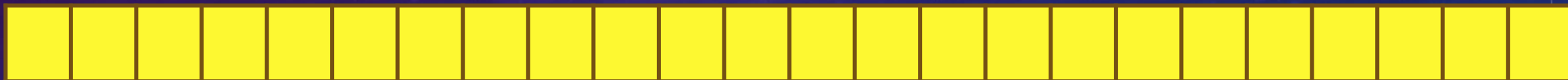
ムダを削ろう

- 「左から個数を足していって、はじめに X_j 以上になる場所はどこ？」という質問に高速に答えたい
- 質問があらかじめ全て分かっているので、 X_j の小さい順に答えるとして、前の質問の答えを利用したい
 - 今回は制約で $X_1 \leq X_2 \leq \dots \leq X_Q$ が保証されているので、並べ替える必要はない



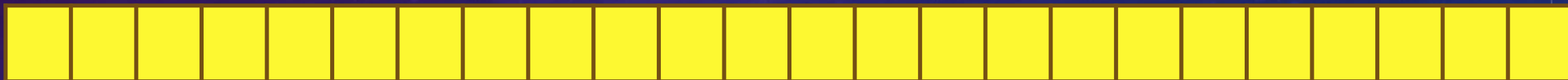
ムダを削ろう

- 「左から個数を足していって、はじめに X_j 以上になる場所はどこ？」という質問に高速に答えたい
- 質問があらかじめ全て分かっているので、 X_j の小さい順に答えるとして、前の質問の答えを利用したい
- 位置とそこまでの個数の和を記録しながら右に進む方針は小課題 2 と同じ
- いちいち最初から始めず、前の答えとなる位置から出発する



ムダを削ろう

- 「左から個数を足していって、はじめに X_j 以上になる場所はどこ？」という質問に高速に答えたい
- 質問があらかじめ全て分かっているので、 X_j の小さい順に答えるとして、前の質問の答えを利用したい
- 位置とそこまでの個数の和を記録しながら右に進む方針は小課題 2 と同じ
- いちいち最初から始めず、前の答えとなる位置から出発する



ステッカー

- この図からイメージできる通り、右に進む回数が N ステップで抑えられる ▶
- これで満点が取れました



解法 2

- 「 $X_1 \leq X_2 \leq \dots \leq X_Q$ 」という保証もなく，1つ答えないと次の質問を教えてくれない場合はどうだろう？
- 小課題 2 の解法を引き継いだ別の方針で解けるので，そちらも紹介します

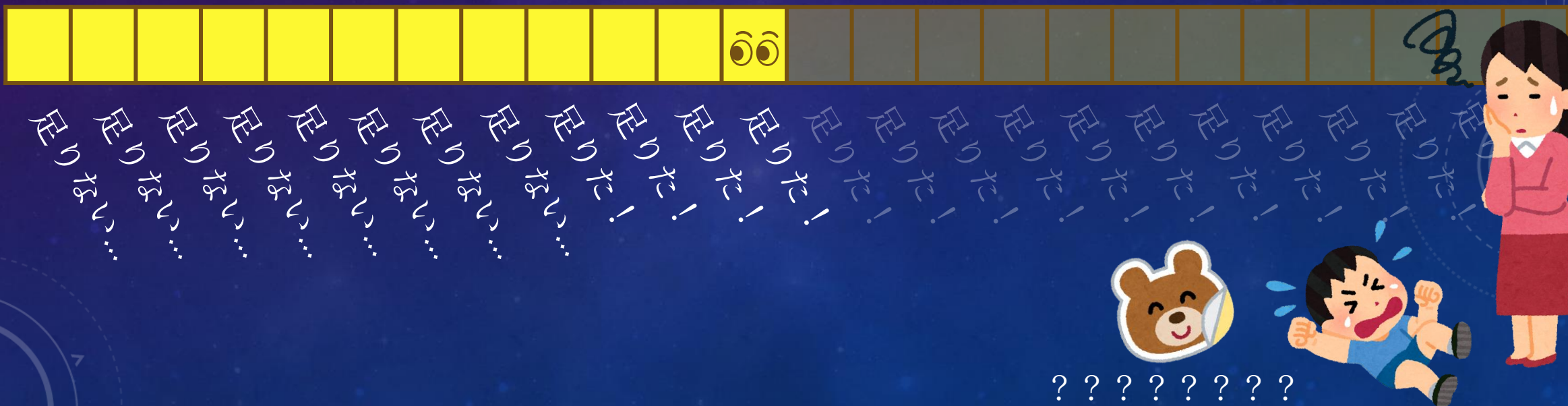
单调性

- 「左から個数を足していったら、はじめに X_j 以上になるところはどこ？」という質問に高速に答えたい
- 質問をよく観察してみると、どんな数値設定でも下図のような状況になる
- このように、「右に行って悪化することがない」ということを「単調性がある」と呼ぶ



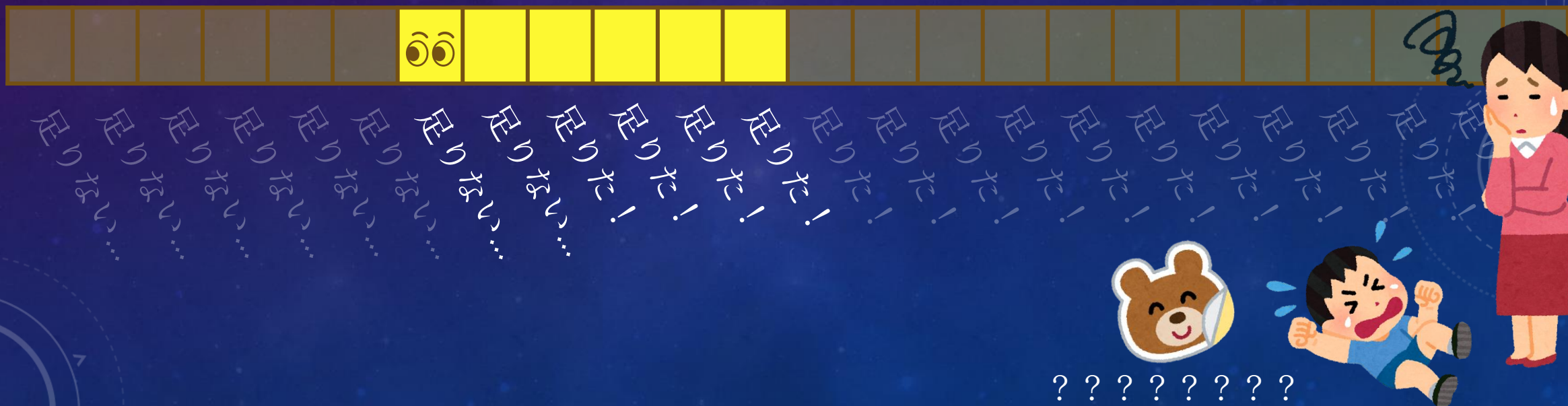
単調性と二分探索

- ド真ん中を見て、そこが「足りた！」ならもうそこより右は見なくていいし、そこが「足りない...」ならもうそこより左は見なくていい



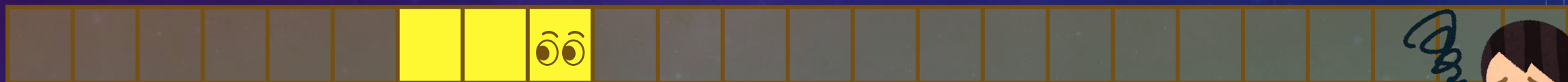
単調性と二分探索

- ド真ん中を見て、そこが「足りた！」ならもうそこより右は見なくていいし、そこが「足りない...」ならもうそこより左は見なくていい
- 一度に半分の範囲に絞れるので、非常に高速に($\log N$ 回ほどで)境界の位置が特定できる



単調性と二分探索

- ド真ん中を見て、そこが「足りた！」ならもうそこより右は見なくていいし、そこが「足りない...」ならもうそこより左は見なくていい
- 一度に半分の範囲に絞れるので、非常に高速に($\log N$ 回ほどで)境界の位置が特定できる
- このテクニックは「二分探索」として知られる



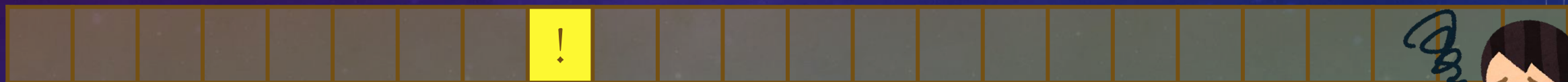
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!



????????

単調性と二分探索

- ド真ん中を見て、そこが「足りた！」ならもうそこより右は見なくていいし、そこが「足りない...」ならもうそこより左は見なくていい
- 一度に半分の範囲に絞れるので、非常に高速に($\log N$ 回ほどで)境界の位置が特定できる
- このテクニックは「二分探索」として知られる



足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!
足りた!



ステッカーカッテ

????????
????????

累積和

- 「左から個数を足していった」結果を各ピースについて保持すれば、それを見て X_j と比べるだけで「足りない」か「足りる」かが判定できる
- この「左から個数を足していった」結果は素直に左から順に計算して記録すればよい



1 個	2 個	1 個	4 個	1 個	1 個	16 個	2 個
累計 1 個	累計 3 個	累計 4 個	累計 8 個	累計 9 個	累計 10 個	累計 26 個	累計 28 個

????????
????????



これで満点が取れました

- 「左から個数を足していった」結果を各ピースについて保持すれば、それを見て X_j と比べるだけで「足りない」か「足りる」かが判定できる
- この「左から個数を足していった」結果は素直に左から順に計算して記録すればよい

1 個	2 個	1 個	4 個	1 個	1 個	16 個	2 個
累計 1 個	累計 3 個	累計 4 個	累計 8 個	累計 9 個	累計 10 個	累計 26 個	累計 28 個

- 累積和に対して「 X_j 以上」という条件で二分探索することで、求める位置がどのピース由来かを特定でき、そのピースの分割後の長さが答えとなる
 - 十分高速になり、満点が得られる



これで満点が取れました

- 「左から個数を足していった」結果を各ピースについて保持すれば、それを見て X_j と比べるだけで「足りない」か「足りる」かが判定できる
- この「左から個数を足していった」結果は素直に左から順に計算して記録すればよい

1 個	2 個	1 個	4 個	1 個	1 個	16 個	2 個
累計 1 個	累計 3 個	累計 4 個	累計 8 個	累計 9 個	累計 10 個	累計 26 個	累計 28 個

- 累積和に対して「 X_j 以上」という条件で二分探索することで、求める位置がどのピース由来かを特定でき、そのピースの分割後の長さが答えとなる
 - 十分高速になり、満点が得られる

得点分布

$$\frac{2}{3} \div \frac{7}{6} = ?$$

- ごちそうさまでした

100 点 166 人	60 点 3 人	35 点 2 人	25 点 5 人
累計 166 人	累計 169 人	累計 171 人	累計 176 人

得点分布

$$\frac{2}{3} \div \frac{7}{6} = 4ラ$$

- ごちそうさまでした
- お後がよろしいようで

100 点 166 人	60 点 3 人	35 点 2 人	25 点 5 人
累計 166 人	累計 169 人	累計 171 人	累計 176 人

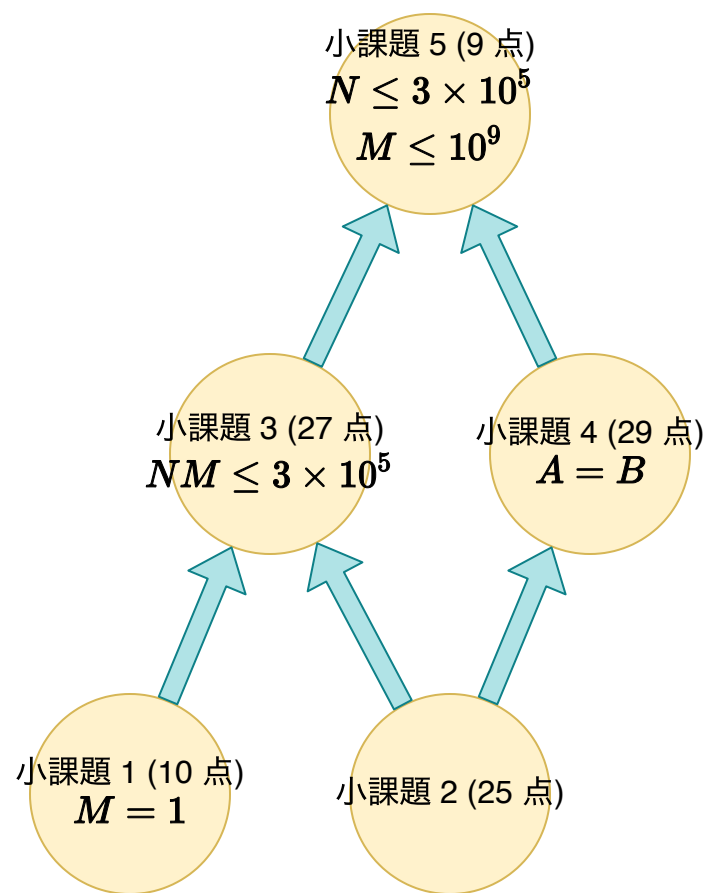
JOI 2021/2022 本選 問題 2 自習 (Self Study) 解説

解説担当 : tatyam

問題概要

- N 科目の授業が、科目 $1, 2, \dots, N, 1, 2, \dots$ と N 日周期で各科目 M 回行われる。
- 科目 i の授業に 1 日出ると、科目 i の理解度が A_i 増える。
- 科目 i の自習を 1 日すると、科目 i の理解度が B_i 増える。
- 授業はその日やっている科目しか受けられないが、自習は好きな科目を勉強できる。
- 1 日にできる行動は 1 つまで。
- 理解度の最小値を最大化せよ。

小課題



小課題 1 (10 点)

- $M = 1$
- N 日間で N 科目の勉強をする。
- 勉強しない科目があれば理解度の最小値は 0
 - 全ての科目を 1 回ずつ勉強する。
 - i 日目に科目 i を勉強する。授業か自習のうち良い方を選ぶ。
- $O(N)$

小課題 2 (25 点)

- $NM \leq 3 \times 10^5$
- $A = B \rightarrow$ 授業は受けなくて良い (!)
- 好きな科目を自習できる $\rightarrow$ 理解度の低い科目を優先的に自習
- 「理解度の最も低い科目を選び自習」を NM 回行えば良い。
- 理解度の最も低い科目を効率的に選ぶには...? $\rightarrow$ ヒープ (priority_queue)
- $O(NM \log N)$

小課題 3 (27 点)

- $NM \leq 3 \times 10^5$

考察 1

- $A_i < B_i$ のとき、授業を自習に置き換えることによって理解度を上げられる。
- $A_i \leftarrow \max(A_i, B_i)$ と更新しておけば、 $A_i \geq B_i$ を仮定できる。

考察 2

- 各科目の勉強は授業でやった方が効率が良い。
- 各科目の勉強を何回行うか決めたとき、そのうち授業は何回受けられるか？
→ 各科目独立に M 回まで受けられる (!)
(各科目 M 回までをその科目の授業の枠に入れた後、残りを余った枠に入れることで達成。)

小課題 3 (27 点)

- 小課題 2 から少し変更したアルゴリズムで解くことができる。
- $A_i \leftarrow \max(A_i, B_i)$ と更新しておく。
- 以下を $N \times M$ 回行う。
 - `priority_queue` で理解度の最も低い科目 i を選ぶ。
 - M 回目までの科目 i の勉強 $\rightarrow$ 理解度が A_i 上がる
 - $M + 1$ 回目以降の科目 i の勉強 $\rightarrow$ 理解度が B_i 上がる
- $O(NM \log N)$

小課題 4 (29 点)

- $A = B \rightarrow$ 授業は受けなくて良い (!)
- $N \leq 3 \times 10^5, M \leq 10^9 \rightarrow$ 小課題 2 の $O(NM \log N)$ 解は TLE

考察

- NM 回自習を行って、理解度の **最小値の最大化**
- 最小値の最大化 $\rightarrow$ 答えを二分探索 !

答えを二分探索

- 最小値の最大値、L 番目に小さい値の K 番目に小さい値、中央値の中央値など、**代表値の代表値** といえは **答えを二分探索**！
- 「答えは x 以上ですか？」という判定問題が高速に解けるとき、答えを二分探索することで元の問題を解くテクニック
- AtCoder Beginner Contest にも頻出
 - [ABC236 E - Average and Median](#)
 - [ABC227 D - Project Planning](#)

小課題 4 (29 点)

- 判定問題: NM 回自習を行って、全ての科目の理解度を x 以上にできますか？
- 科目 i の理解度を x 以上にするには $\left\lceil \frac{x}{B_i} \right\rceil$ 回の自習が必要
- $\sum_i \left\lceil \frac{x}{B_i} \right\rceil \leq NM$ であれば Yes、そうでなければ No
- 判定問題が $O(N)$ で解けるので、二分探索の上限を $MB_{\max}$ とすると $O(N \log(MB_{\max}))$ でこの小課題が解ける。

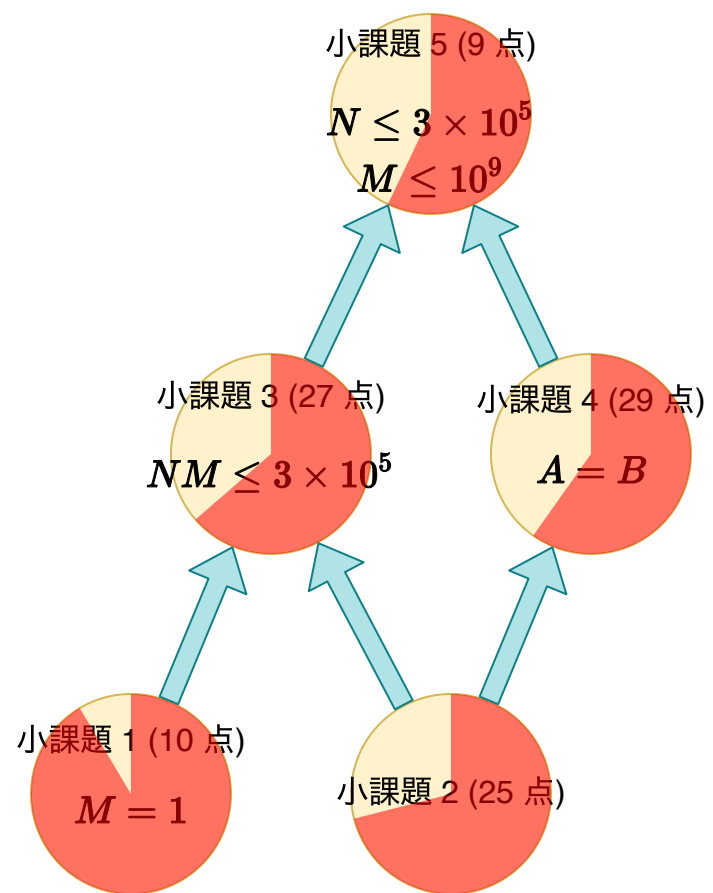
小課題 5 (9 点)

- 追加の制約はない
- 小課題 3 と 4 を組み合わせよう
- 小課題 3 の考察：各科目の勉強を何回行うか決めたとき、そのうち授業は各科目独立に M 回まで受けられる
- 小課題 4 の考察：「全ての科目の理解度を x 以上にできますか？」という判定問題を解いて答えを二分探索

小課題 5 (9 点)

- 判定問題: NM 回自習や授業を行って、全ての科目の理解度を x 以上にできますか？
- $A_i \geq B_i$ と仮定する。
- 科目 i の理解度を x 以上にするには...
 - $x \leq MA_i$ ならば、 $\left\lceil \frac{x}{A_i} \right\rceil$ 回の授業
 - そうでなければ、 M 回の授業と $\left\lceil \frac{x - MA_i}{B_i} \right\rceil$ 回の自習が必要
- 判定問題が $O(N)$ で解けるので、二分探索の上限を $MA_{\max}$ とすると $O(N \log(MA_{\max}))$ で満点を得られる。 100

得点分布



第3問 「選挙で勝とう」

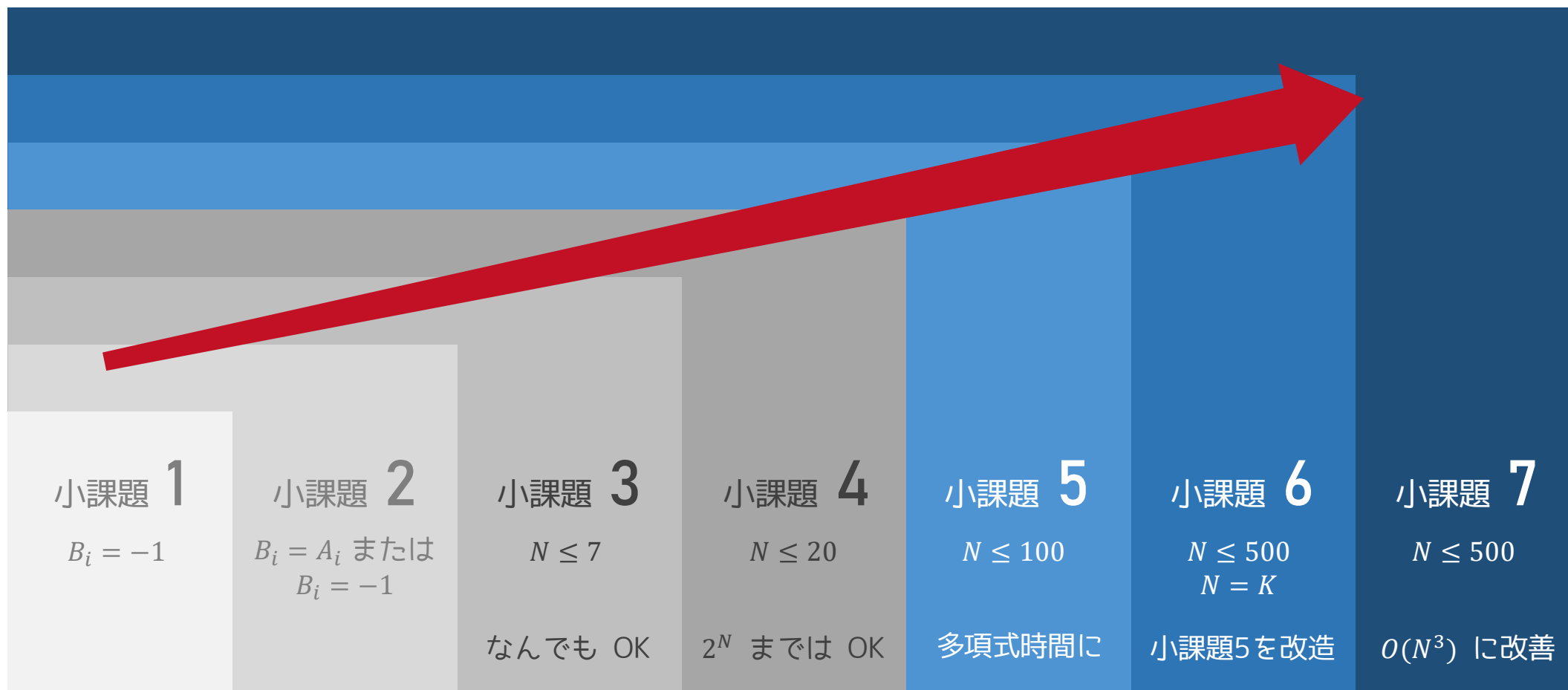
解説：米田 優峻（E869120）



- N 個の州で選挙活動をします
 - 州 i では、合計演説時間が A_i 時間に達すると票が得られます
 - 州 i では、合計演説時間が B_i 時間に達すると協力者が得られます
- 合計 K 票得るために、最小で何時間必要ですか？



演説をする理恵さん



小課題 1

$$B_i = -1$$

- $B_i = -1$ なので、協力者がいない
 - したがって、全部一人で演説する必要がある
- 票を得るのにかかる時間が短い州から取っていくと良い

- $B_i = -1$ なので、協力者がいない
 - したがって、全部一人で演説する必要がある
- 票を得るのにかかる時間が短い州から取っていくと良い

州 1	州 2	州 3	州 4	州 5	州 6	州 7
4時間	11時間	6時間	12時間	36時間	11時間	20時間

たとえば左の例で $K = 4$ 票集めたい場合、票を取るのが簡単な州 1, 2, 3, 6 を取れば良い

かかる時間は $4 + 11 + 6 + 11 = 32$ 時間

```
#include <iostream>
using namespace std;

int N, K;
int A[509], B[509];

int main() {
    // 入力
    cin >> N >> K;
    for (int i = 0; i < N; i++) {
        cin >> A[i] >> B[i];
    }

    // ソート
    sort(A, A + N);

    // 答えを出力
    int Answer = 0;
    for (int i = 0; i < K; i++) {
        Answer += A[i];
    }
    cout << Answer << endl;
    return 0;
}
```

- 左のように、**ソート**を使うと簡単に実装することができます
- 皆さん、JOI では取れる部分点を全部取りましょう。

5点

小課題 2

$$B_i = -1 \text{ または } B_i = A_i$$

- $B_i = -1$ または $B_i = A_i$
 - 協力者がいない(タイプ A)、または票と協力者が同時に得られる(タイプ B)
- このようなケースでは、次の 3 つのことがいえる：
 - タイプ A の中では、簡単な州 [A_i が小さい州] から取った方が良い

- $B_i = -1$ または $B_i = A_i$
 - 協力者がいない(タイプ A)、または票と協力者が同時に得られる(タイプ B)
- このようなケースでは、次の 3 つのことがいえる：
 - タイプ A の中では、簡単な州 [A_i が小さい州] から取った方が良い
 - タイプ B の中では、簡単な州 [A_i が小さい州] から取った方が良い

- $B_i = -1$ または $B_i = A_i$
 - 協力者がいない(タイプ A)、または票と協力者が同時に得られる(タイプ B)
- このようなケースでは、次の 3 つのことがいえる：
 - タイプ A の中では、簡単な州 [A_i が小さい州] から取った方が良い
 - タイプ B の中では、簡単な州 [A_i が小さい州] から取った方が良い
 - 協力者は早く得た方が良いので、タイプ B の州から演説をすべき

- そこで、タイプ B から取る票の数 $Goal$ を全探索する
 - $Goal$ が決まったら、まずタイプ B の州を A_i の小さい順に $Goal$ 個取る
 - 次に、タイプ A の州を A_i の小さい順に $K - Goal$ 個取る

- そこで、**タイプ B から取る票の数 $Goal$ を全探索**する
 - $Goal$ が決まったら、まずタイプ B の州を A_i の小さい順に $Goal$ 個取る
 - 次に、タイプ A の州を A_i の小さい順に $K - Goal$ 個取る

	州 1	州 2	州 3	州 4	州 5
票獲得まで A_i	4	5	6	7	8
協力者まで B_i	-	-	-	7	8

左の例で $K = 3$ 票取することを考える

$Goal = 0$ の場合は...

- 州 1 $\rightarrow$ 2 $\rightarrow$ 3 の順に取れば良い
- $4 + 5 + 6 = 15$ 時間かかる

- そこで、**タイプ B から取る票の数 $Goal$ を全探索**する
 - $Goal$ が決まったら、まずタイプ B の州を A_i の小さい順に $Goal$ 個取る
 - 次に、タイプ A の州を A_i の小さい順に $K - Goal$ 個取る

	州 1	州 2	州 3	州 4	州 5
票獲得まで A_i	4	5	6	7	8
協力者まで B_i	-	-	-	7	8

左の例で $K = 3$ 票取することを考える

$Goal = 1$ の場合は…

- 州 4 $\rightarrow$ 1 $\rightarrow$ 2 の順に取れば良い
- $7 + 4/2 + 5/2 = 11.5$ 時間かかる

- そこで、**タイプ B から取る票の数 $Goal$ を全探索**する
 - $Goal$ が決まったら、まずタイプ B の州を A_i の小さい順に $Goal$ 個取る
 - 次に、タイプ A の州を A_i の小さい順に $K - Goal$ 個取る

	州 1	州 2	州 3	州 4	州 5
票獲得まで A_i	4	5	6	7	8
協力者まで B_i	-	-	-	7	8

左の例で $K = 3$ 票取することを考える

$Goal = 2$ の場合は…

- 州 4 $\rightarrow$ 5 $\rightarrow$ 1 の順に取れば良い
- $7 + 8/2 + 4/3 = 12.3$ 時間かかる

- そこで、タイプ B から取る票の数 $Goal$ を全探索する
 - $Goal$ が決まったら、まずタイプ B の州を A_i の小さい順に $Goal$ 個取る
 - 次に、タイプ A の州を A_i の小さい順に $K - Goal$ 個取る

$Goal = 0, 1, 2$ の中で最も時間が少ない

11.5 時間が答え

	州 1	州 2	州 3	州 4	州 5
票獲得まで A_i	4	5	6	7	8
協力者まで B_i	-	-	-	7	8

この州で $K = 3$ 票取することを考える

$Goal = 2$ の場合は...

- 州 4 $\rightarrow$ 5 $\rightarrow$ 1 の順に取れば良い
- $7 + 8/2 + 4/3 = 12.3$ 時間かかる

2 小課題 2 の実装

17 / 47

```
#include <bits/stdc++.h>
using namespace std;
int N, K, A[509], B[509];
vector<int> vec1, vec2;

int main() {
    // 入力
    cin >> N >> K;
    for (int i = 0; i < N; i++) {
        cin >> A[i] >> B[i];
        if (B[i] == -1) vec1.push_back(A[i]);
        if (B[i] == A[i]) vec2.push_back(A[i]);
    }

    // ソート
    sort(vec1.begin(), vec1.end());
    sort(vec2.begin(), vec2.end());

    // 答えを求める
    double Answer = 1e9;
    for (int Goal = 0; Goal <= K; Goal++) {
        if (K - Goal > vec1.size() || Goal > vec2.size()) continue;
        double sum = 0;
        for (int i = 0; i < K - Goal; i++) sum += 1.0 * vec1[i] / (Goal+1);
        for (int i = 0; i < Goal; i++) sum += 1.0 * vec2[i] / (i+1);
        Answer = min(Answer, sum);
    }
    printf("%.12lf¥n", Answer);
    return 0;
}
```

- したがって、左のような実装をすれば良い
- $Goal$ の値を $O(N)$ 通り全探索しているので、計算量は $O(N^2)$
- $N \leq 500$ なので間に合う

10点

小課題 3

$$N \leq 7$$

- 各州について、最適解は明らかに以下の 3 つのいずれか：
 - パターン A: 0 時間演説する
 - パターン B: A_i 時間演説する (票を得る)
 - パターン C: B_i 時間演説する (協力者を得る)
- 各州の演説時間が決まれば、最適なムーブはすぐに分かる
 - 小課題 2 と同様、協力者を得てから票を得るのが最適

3 小課題 3 の解法

20 / 47

- そこで、各州の演説時間を全探索することを考える
 - 全部で $3 \times 3 \times \dots \times 3 = 3^N$ 通りある
 - $N \leq 7$ なので、小課題 3 が通る
- 制約が小さいため、「協力者を得てから票を得る」という考察をせずに演説の順番を全探索しても通る

21点

小課題 4

$$N \leq 20$$

- 「**どの州で協力者を得るか？**」を全探索することを考える
 - 全部で 2^N 通りを調べる必要がある
- 協力者を得る州が決まったら…
 - あとは明らかに **A_i の小さい順**に票を得れば良い
 - したがって、「0 時間か A_i 時間か」を全探索せずとも演説をする州が決まる

4

小課題 4 の解法

23 / 47

たとえば以下の例を考えよう ($K = 4$ 票得れば良い)

	州 1	州 2	州 3	州 4	州 5	州 6
票獲得まで A_i	3時間	5時間	6時間	4時間	7時間	2時間
協力者まで B_i	3時間	6時間	6時間	8時間	9時間	12時間

4

小課題 4 の解法

24 / 47

たとえば以下の例を考えよう ($K = 4$ 票得れば良い)

	州 1	州 2	州 3	州 4	州 5	州 6
票獲得まで A_i	3時間	5時間	6時間	4時間	7時間	2時間
協力者まで B_i	3時間	6時間	6時間	8時間	9時間	12時間

州 1 と州 3 で協力者を得たとする

たとえば以下の例を考えよう ($K = 4$ 票得れば良い)

	州 1	州 2	州 3	州 4	州 5	州 6
票獲得まで A_i	3時間	5時間	6時間	4時間	7時間	2時間
協力者まで B_i	3時間	6時間	6時間	8時間	9時間	12時間

州 1 と州 3 で協力者を得たとする

- 残り 2 票は、 A_i の値が小さい州 4 と州 6 で演説をすれば良い
- 合計時間は $3 + 6/2 + 4/3 + 2/3 = 8$ 時間

4

小課題 4 の解法

26 / 47

たとえば以下の例を考えよう ($K = 4$ 票得れば良い)

	州 1	州 2	州 3	州 4	州 5	州 6
票獲得まで A_i	3時間	5時間	6時間	4時間	7時間	2時間
協力者まで B_i	3時間	5時間	6時間	8時間	9時間	12時間

ビット全探索などを用いて「協力者を得る州」を

全て調べれば良い！

州 1 と州 3 で協力者を得るとする
計算量は $O(2^N \times N \log N)$

- 残り 2 票は、 A_i の値が小さい州 4 と州 6 で演説をすれば良い
- 合計時間は $3 + 6/2 + 4/3 + 2/3 = 8$ 時間

※ここまで 33 点

小課題 5

$$N \leq 100$$

小課題 2 の解法を
思い出してみましょう

- 小課題 2 では、最終的な協力者の人数 $Goal$ を全探索した
 - 整数 $Goal$ が決まった場合のメリットは…？
 - 「票しか得ない州」に必要な演説時間も定まる
- A_i, B_i が一般のケースでも、 $Goal$ の値が決まると DP とかに落とせたりしないのか？

以下のアルゴリズムにより、計算量 $O(N^4)$ が実現できる

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか}][\text{得た票の数}][\text{協力者の人数}] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

以下のアルゴリズムにより、計算量 $O(N^4)$ が実現できる

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか}][\text{得た票の数}][\text{協力者の人数}] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

以下のアルゴリズムにより、計算量 $O(N^4)$ が実現できる

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか}][\text{得た票の数}][\text{協力者の人数}] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

以下のアルゴリズムにより、計算量 $O(N^4)$ が実現できる

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか}][\text{得た票の数}][\text{協力者の人数}] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

演説をしない場合 $dp[i + 1][j][k] := dp[i][j][k]$

票だけ取る場合 $dp[i + 1][j + 1][k] := dp[i][j][k] + A_i / (Goal + 1)$ ← 票だけ取る州は最後に取り

協力者まで取る場合 $dp[i + 1][j + 1][k + 1] := dp[i][j][k] + B_i / (k + 1)$ ← 協力者は B_i の小さい順に取り

- 計算量を考えてみよう
 - 協力者の人数 $Goal$ は $O(N)$ 通り全探索する
 - DP は三次元配列を使うので、状態数が $O(N^3)$ 通り
- 全体の計算量は $O(N) \times O(N^3) = O(N^4) \rightarrow N \leq 100$ なら間に合う
 - 過半数獲得おめでとう！

小課題 6

$$N = K$$

- $N = K$ なので、「0 時間演説する」という選択肢はない
 - そのため、**暫定獲得票数は DP 配列にメモしなくて良い**
- 協力者の人数 x を全探索したうえで、次のような DP をすれば良い
 - $\text{dp}[\text{何番目の州まで決めたか}][\text{協力者の人数}] = \text{時間の最小値}$
 - ただし、小課題 5 と同様 B_i の昇順にソートする必要がある

6 小課題 6 の解法

37 / 47

- 実装は小課題 5 を少し変更するだけで良い
 - DP の「暫定獲得票数」の次元を削り、票を得ない選択肢を消すだけ
- 計算量は $O(N^3)$ なので、 $N \leq 500$ でも間に合う
 - 3 分の 2 獲得おめでとう！

77点



小課題 7

追加の制約はない

- 簡単のため、 $B_1 \leq B_2 \leq \dots \leq B_N$ を仮定する
- そのとき、次のような組 $x < y$ があるのは絶対に損
 - 州 x で演説をせず、州 y で B_y 時間演説して協力者を得る

- 簡単のため、 $B_1 \leq B_2 \leq \dots \leq B_N$ を仮定する
- そのとき、次のような組 $x < y$ があるのは**絶対に損**
 - 州 x で演説をせず、州 y で B_y 時間演説して協力者を得る

なぜか？ → 州 y の代わりに州 x で協力者を得た方が時間が短い！

	州1	州2	州3	州4	州5
票獲得まで A_i	1	6	4	4	8
協力者まで B_i	3	6	7	9	11



	州1	州2	州3	州4	州5
票獲得まで A_i	1	6	4	4	8
協力者まで B_i	3	6	7	9	11

※青は票獲得だけ、赤は協力者まで

7

小課題 7 の解法

41 / 47

- 簡単のため、 $B_1 \leq B_2 \leq \dots \leq B_N$ を仮定する
- そのとき、次のような組 $x < y$ があるのは絶対に損
 州 $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow N$ の順に決めていったとき
 - 州 x で演説をせず、州 y で B_x 時間演説して協力者を得る
 協力者を目標の x 人集めるまでは

なぜか？ → 州 y の代わりに州 x で協力者を得た方が時間が短い！

演説しないという選択肢はない！

	州1	州2	州3	州4	州5
票獲得まで A_i	1	6 ← 4	4	4	8
協力者まで B_i	3	6 ← 7	9	9	11

※青は票獲得だけ、赤は協力者まで

そこで小課題 5 の解法を振り返ってみよう

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか } i][\text{得た票の数 } j][\text{協力者の人数 } k] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

そこで小課題 5 の解法を振り返ってみよう

手順 1 協力者を得るまでにかかる時間 B_i の昇順にソートする

手順 2 協力者の人数 $Goal$ を決めた上で、次のような DP をする

- $dp[\text{何番目の州まで決めたか } i][\text{得た票の数 } j][\text{協力者の人数 } k] = \text{時間の最小値}$
- 遷移は、州の演説時間が $\{0, A_i, B_i\}$ のどれであるかを一つずつ決めていく感じ

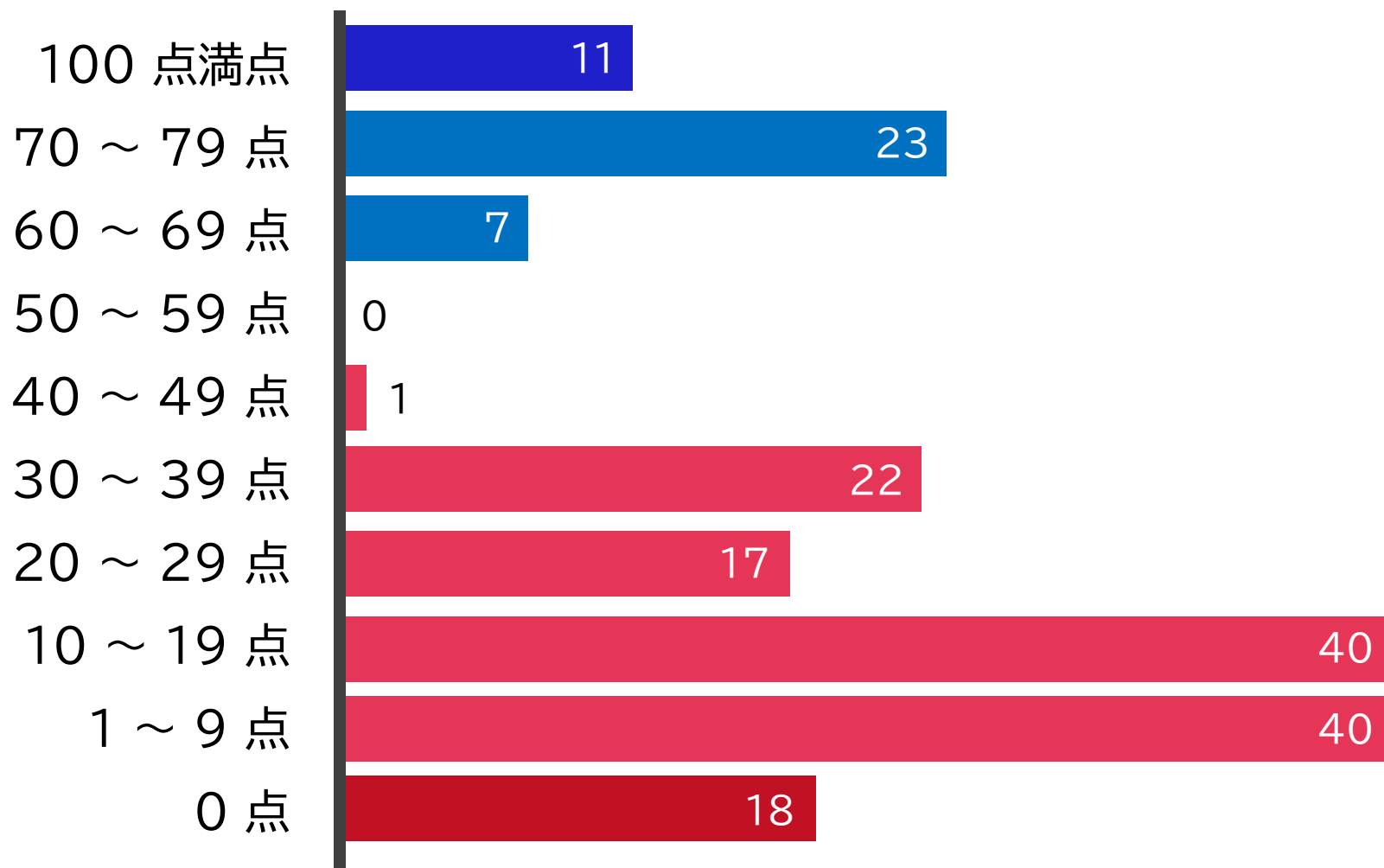
➡ 協力者の人数が $Goal - 1$ 以下の場合は、 $i = j$ が成り立つ

※ $i \neq j$ の場合、協力者が全員集まる前に「演説しないこと」を選んでしまう！

- 考えるべき $dp[i][j][k]$ の状態数はいくつあるのか？
 - i : 何番目の州まで決めたか ($0 \leq i \leq N$)
 - j : 得た票の数 ($0 \leq j \leq K$)
 - k : 得た協力者の人数 ($0 \leq k \leq Goal$ 、 $Goal$ は最終的な協力者数)
- 実は全部で $O(N^2)$ 通りしか考える必要がない
 - $k = Goal$ のとき: 全体で高々 $(N + 1)(K + 1)$ 通り
 - $k < Goal$ のとき: $i = j$ なので各 k ($0 \leq k < Goal$) につき高々 N 通り

- $Goal$ を $0 \leq Goal \leq N$ の範囲で全探索するので、計算量は $O(N^3)$
 - 上手に実装すれば $N \leq 500$ でも通る
 - 全議席獲得おめでとう！
- サンプルコードはメモリ使用量 $O(N^3)$ の実装となっていますが、DP 配列を 2 つ用意するなどすれば、 $O(N^2)$ に減らすことも可能です

得点分布



50点以上 41

50点未満 138

平均点 28.5

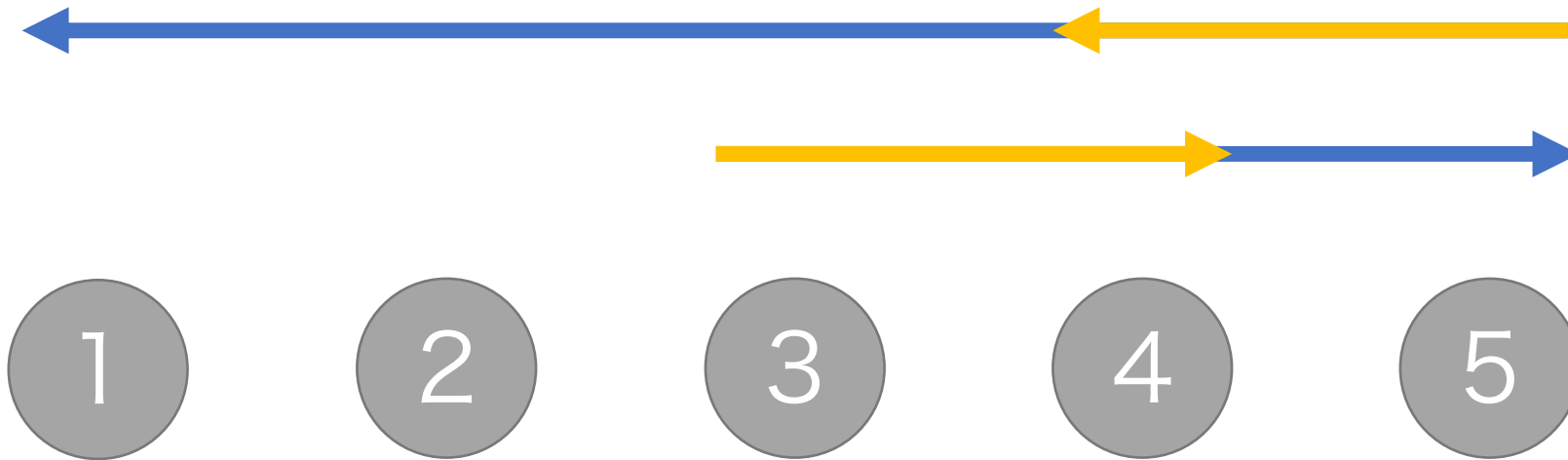


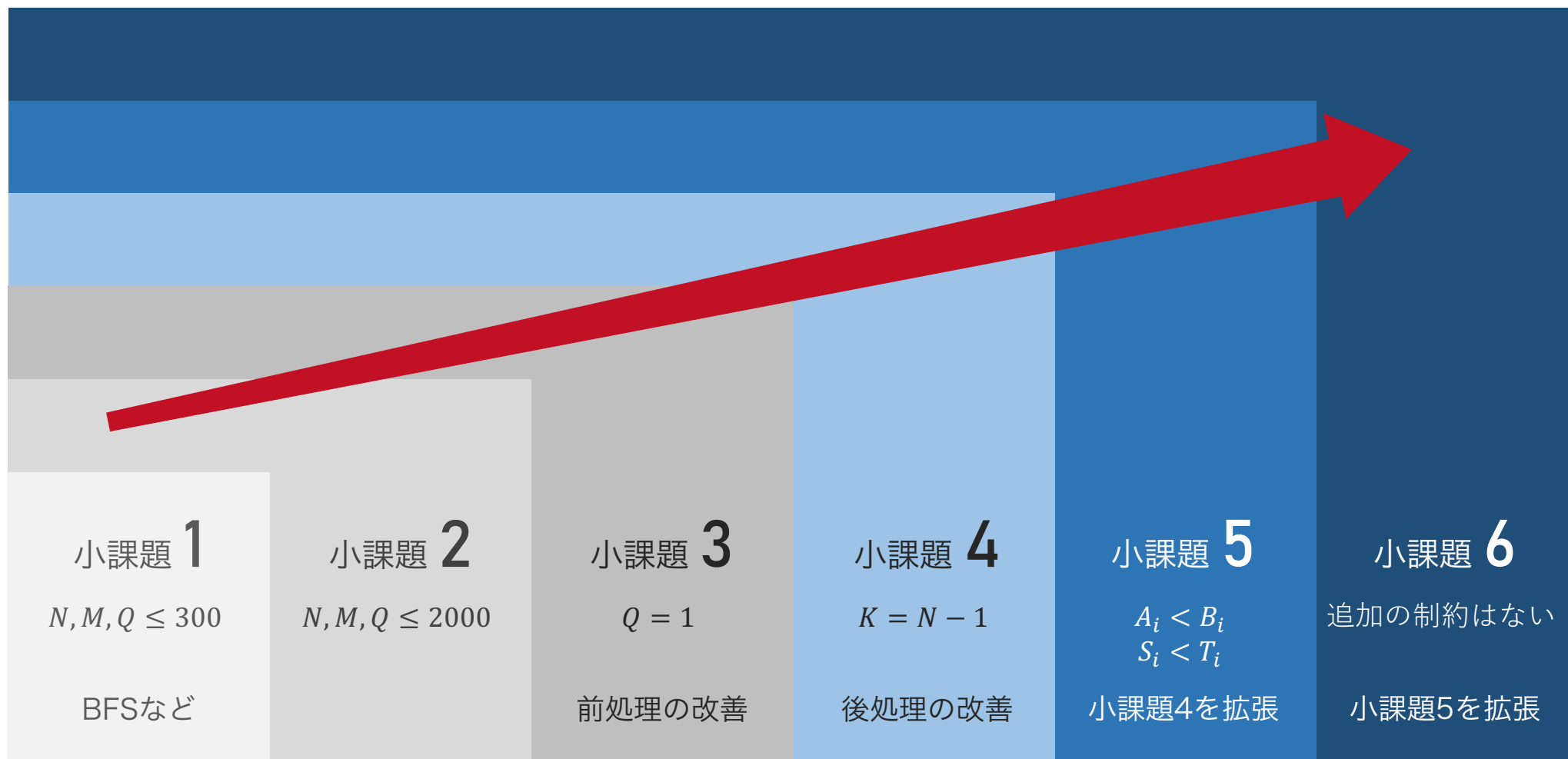
第4問 「鉄道旅行 2」

解説：星井 智仁

- M 本の系統があります
 - 駅 A_i を出発し終点の駅 B_i まで各駅に停車する
 - 駅 A_i を含めた K 駅以内の駅からしか乗車できない
- Q 個のクエリが与えられる
 - 駅 S_i から駅 T_i に訪れるために乗る必要がある列車の本数の最小値は？

- 入力例 1 の図示





小課題 1

$$N, M, Q \leq 300$$

- $N, M, Q \leq 300$ なので、3乗まで許される
 - いきなり多項式時間に落とさないといけない
 - どうしよう

- $N, M, Q \leq 300$ なので、3乗まで許される
 - いきなり多項式時間に落とさないといけない
 - 各クエリあたり $O(NM)$ とか $O(N^2)$ くらいでなんとかしたい

- 各クエリあたり $O(NM)$ で解く 解法
 - (今いる座標, 今乗っている路線) を index にして 01BFS
 - 01BFS は BFS の拡張で, コストが 0 と 1 のときに使える手法
 - Dijkstra 法だと $\log$ が乗るので 厳しいかも

小課題 2

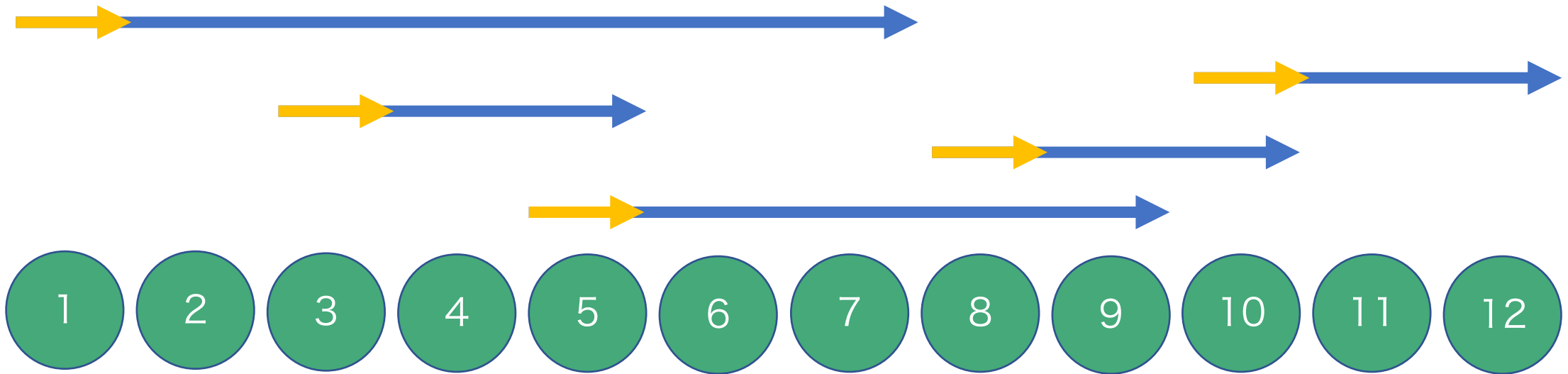
$N, M, Q \leq 2000$

- $N, M, Q \leq 2000$ なので, 2乗まで許される
 - 1つのクエリあたり $O(N)$ とか遅くとも $O(N \log N)$ くらいで答えたい

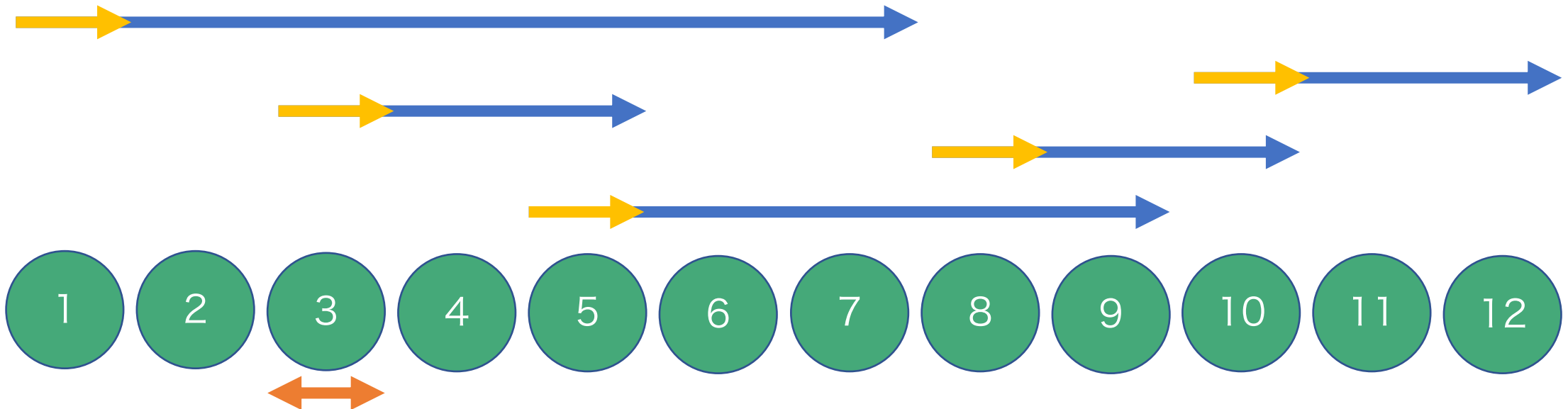
- $N, M, Q \leq 2000$ なので, **2乗**まで許される
 - 1つのクエリあたり $O(N)$ とか遅くとも $O(N \log N)$ くらいで答えたい
- 小課題1から M 倍早くする必要あり
 - どの系統の列車に乗車しているかという情報は必要か?
 - いない
 - 到達可能な区間だけを気にすればよい

- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新

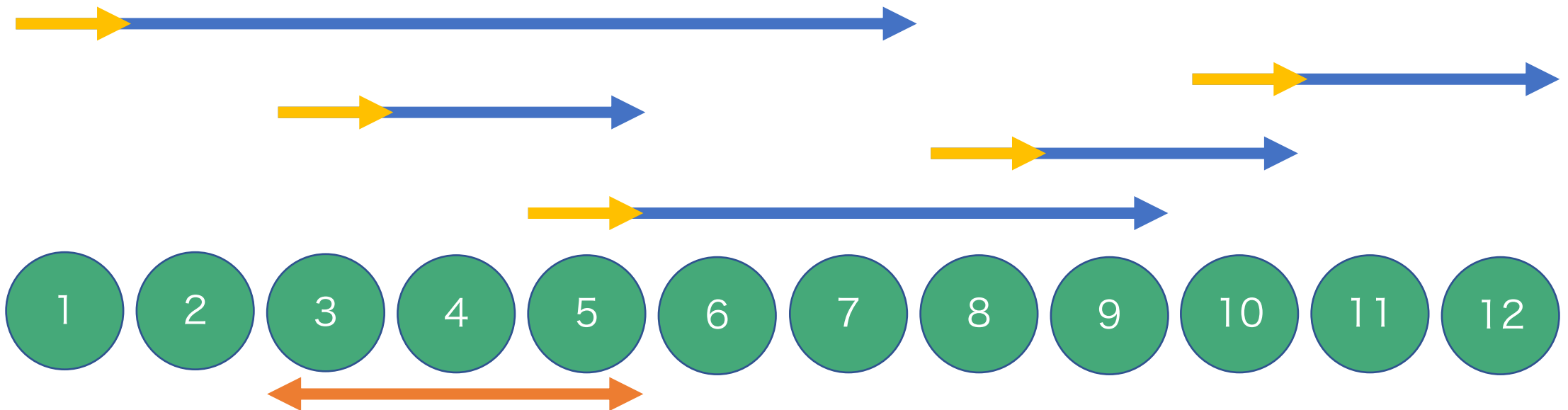
- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



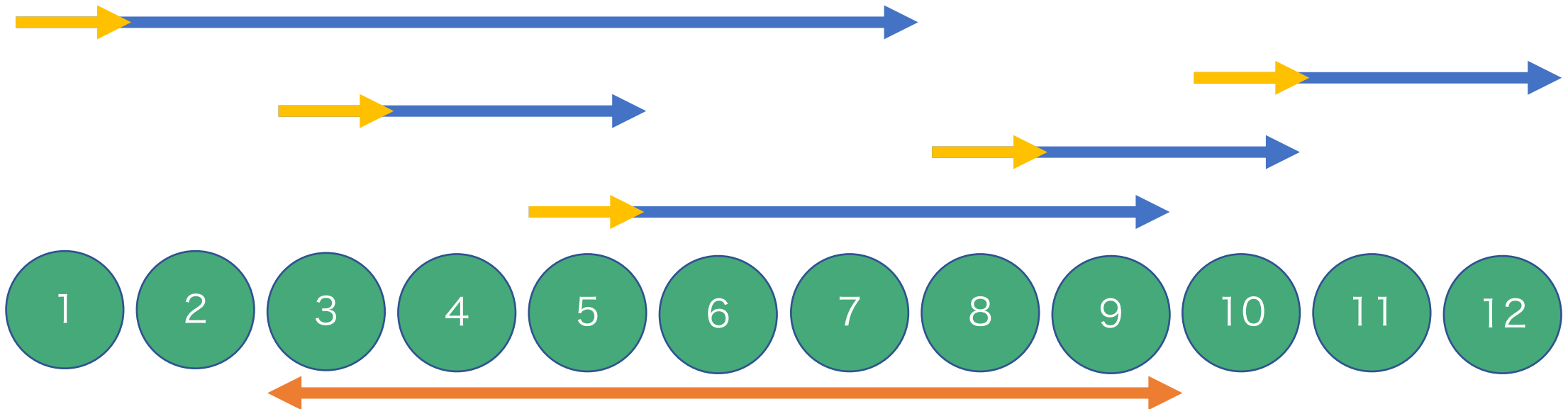
- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



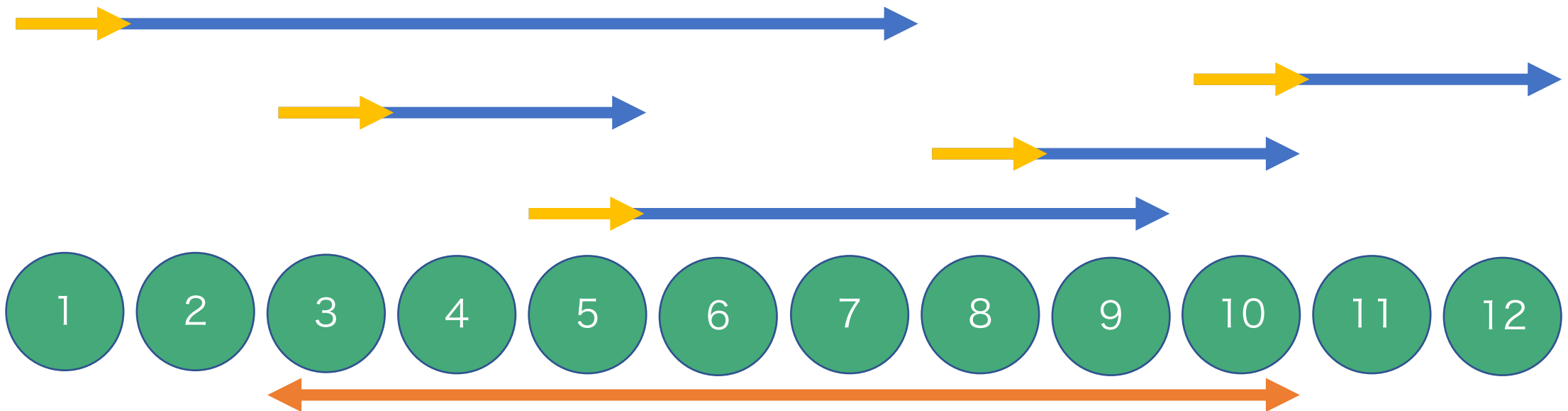
- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



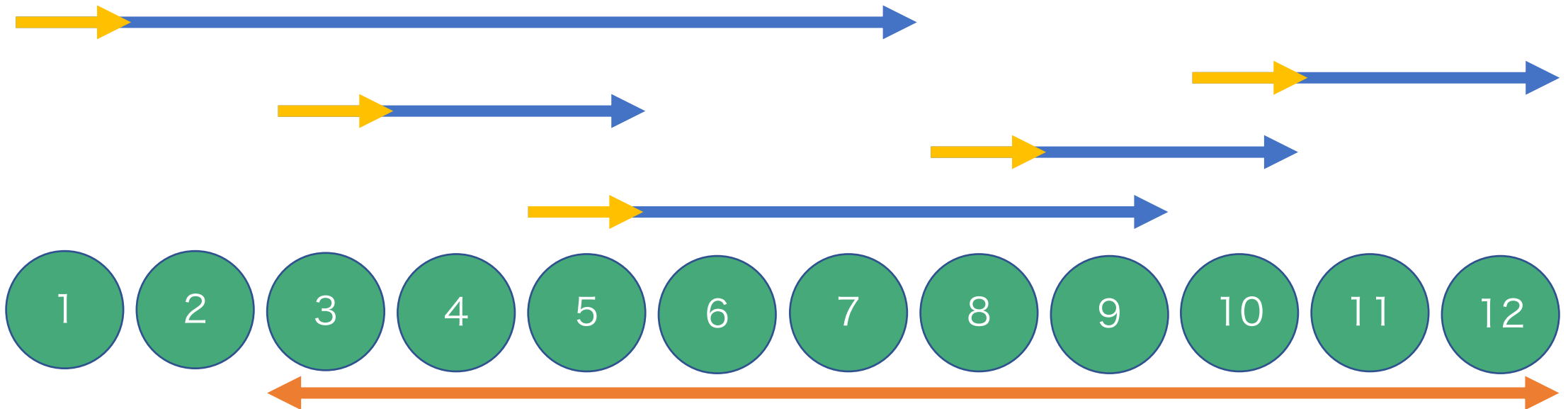
- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



- それぞれの駅から 1 回列車に乗車することで到達可能な範囲は？
 - $O(MK)$ で愚直に前処理して OK
- 到達可能な区間を広げていくように更新



- 前処理 $O(MK)$ 各クエリごとに $O(N)$
- 合計 $O(MK + NQ)$ で解けました

16点



小課題 3

$$Q = 1$$

- 小課題 2 の解法は $O(MK + NQ)$

- 小課題 2 の解法は $O(MK + NQ)$
- 頑張って定数倍高速化をすると解けるらしい

27点

小課題 4

$$K = N - 1$$

- 小課題 2 の解法は $O(MK + NQ)$

- 小課題 2 の解法は $O(MK + NQ)$
 - これを改善したい
 - $Q = 1$ なので, 今のままだと計算量は $O(MK + N)$

- 小課題 2 の解法は $O(MK + NQ)$
 - これを改善したい
 - $Q = 1$ なので, 今のままだと計算量は $O(MK + N)$
 - ボトルネックは $O(MK)$ の前処理

- 前処理の高速化
 - データ構造を使う
 - Segment Treeで $O(M \log N)$
 - スライド最小値で $O(N + M)$
 - お好きなのでどうぞ

27点

小課題 4

$$K = N - 1$$

- $K = N - 1$ が意味するものとは？
 - 列車にどの駅からも乗れる

- $K = N - 1$ が意味するものとは？
 - 列車に **どの駅からも** 乗れる
 - わざわざ遠回りしない
 - 右に行くなら右へ行く列車だけ気にすればよい

- $K = N - 1$ が意味するものは？
 - 列車にどの駅からも乗れる
 - わざわざ遠回りしない
 - 右に行くなら右へ行く列車だけ気にすればよい
 - たどり着けるなるべく遠くの駅まで行く列車に乗るのが最適

- $K = N - 1$ が意味するものは？
 - 列車に **どの駅からも** 乗れる
 - わざわざ遠回りしない
 - 右に行くなら右へ行く列車だけ気にすればよい
 - たどり着けるなるべく遠くの駅まで行く列車に乗るのが最適
 - この手の問題はダブリングで解ける
 - 類題： [ARC060 E - 高橋君とホテル](#)
 - $O((N + Q) \log N)$

27点

25点

- 各場所から1回, 2回, 2^2 回, $\dots$, 2^k 回操作後の場所を記録しておくことで, 各クエリに対して対数オーダーで答える手法
- 記録する際もそれまで計算した結果を用いる
 - 例えば, ある場所から 2^k 回操作後の場所を計算するとき
 - k が小さい方から計算していく
 - 「ある場所から 2^{k-1} 回操作後の場所」から 2^{k-1} 回操作後の場所は既に計算済み
- 説明する時間がないので各自調べてください

小課題 5

$$A_i < B_i \quad S_i < T_i$$

- 制約より，小課題 4 と同様に一方向に進む場合のみ考える

- 制約より，小課題 4 と同様に一方向に進む場合のみ考える
 - 小課題 4 と違い，途中下車するのが最適な場合がある
 - どうしよう

- 制約より，小課題 4 と同様に**一方向に進む場合**のみ考える
 - 小課題 4 と違い，**途中下車**するのが最適な場合がある
 - 小課題 4 と同じ方法が使えないか？
 - 実は使えます

- 途中下車するダブリング
 - ある区間に対して 2^k 回操作した後にたどり着ける最大値を知りたい
 - 区間取得といえば, Segment Tree
 - Sparse Tableでも可
 - $O((N + Q)(\log N)^2)$ とかで解ける

27点

25点

35点

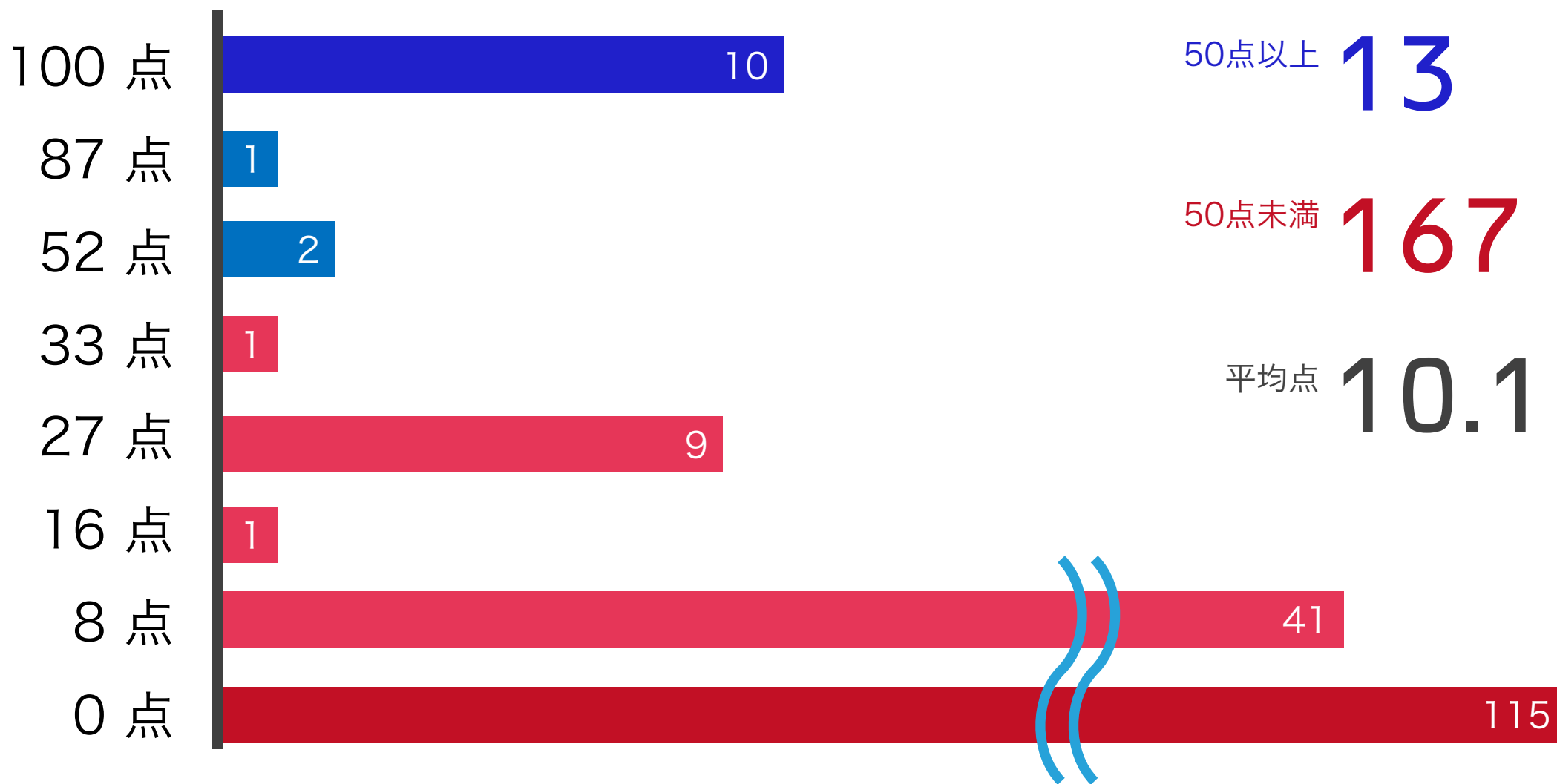
小課題 6

追加の制約はない

- 小課題 5 を双方向にする
 - 右に行ったり左に行ったりしても同様に解ける

100点

得点分布





問題 5: Sandcastle 2

JOI 2021/2022 本選 解説

東京大学一年 米田寛峻 (square1001)



みなさん、競技お疲れ様でした！

問題の説明

$H \times W$ のマス目があり、各マスには整数 $A_{i,j}$ が書かれています

- ただし $A_{1,1}, A_{1,2}, \dots, A_{H,W}$ はすべて相異なる

14	10	7	8	9
2	1	6	12	11
3	4	5	13	15

以下のような長方形領域の個数を数えましょう！

(条件) あるマスから出発し、訪れるマスの整数が単調減少になるように、領域中のすべてのマスを訪れることができる。

14	→	10	→	7	8	9
2	→	1		6	12	11
	↑			↓		
3	←	4	←	5	13	15

14		10		7	→	8	→	9
2		1		6		12	←	11
				↑				↓
3		4		5		13	→	15

14		10		7		8		9
2		1		6		12		11
3	→	4	→	5	→	13	→	15

14		10		7		8		9
2		1		6		12		11
		↓		↑				
3		4	→	5		13		15

14		10		7		8		9
2		1		6		12		11
3		4		5		13		15

...

各小課題の制約

[小課題 1 (9 点)]

$$H = 1$$



[小課題 2 (10 点)]

$$HW \leq 100$$



[小課題 3 (5 点)]

$$HW \leq 1500$$



[小課題 4 (56 点)]

$$HW \leq 7000$$



[小課題 5 (20 点)]

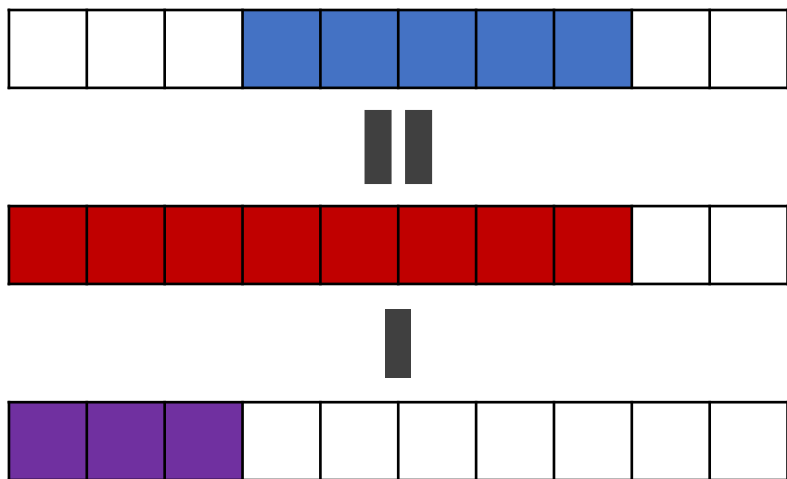
$$HW \leq 50000$$



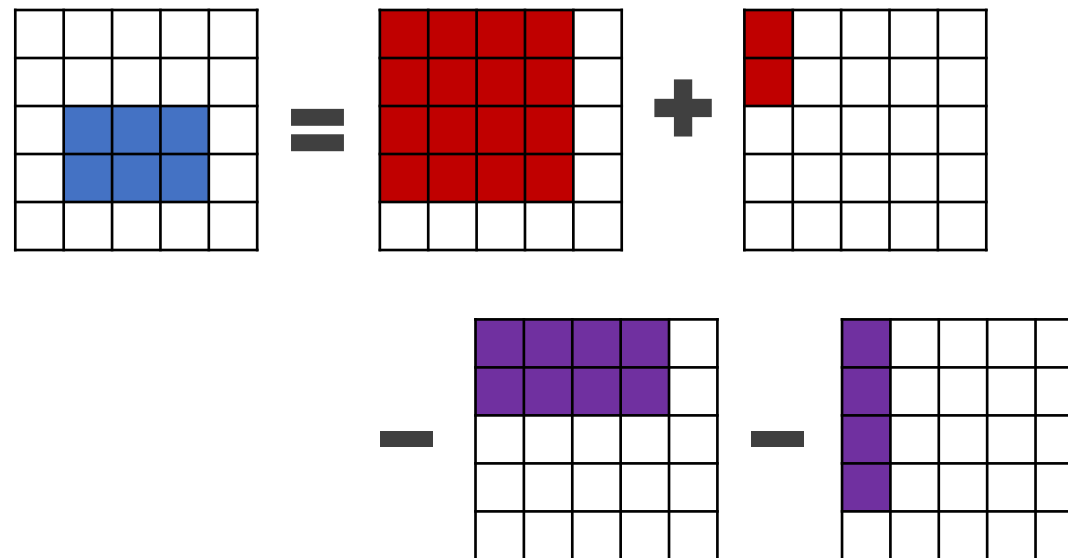
前提知識

この問題は難しいですが、難しい前提知識は必要としません
累積和 を知っていれば解法が理解できます

<1 次元累積和>



<2 次元累積和>



小課題 1 の解法

$H = 1$ の場合、左から右 or 右から左 に進むことに



したがって、この問題の答えは

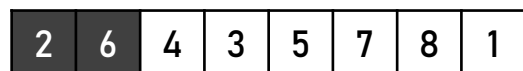
(長さ 2 以上の増加部分列の個数) + (長さ 2 以上の減少部分列の個数) + N

と求められる

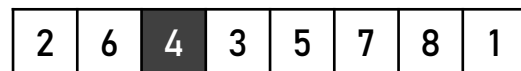
小課題 1 の解法

単調増加な連続部分列 の個数を求めたい

増加する区間に分けて考えると、以下のように求められる



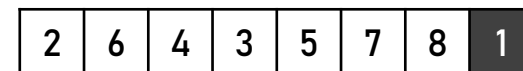
この区間の増加部分列は 1 個



この区間の増加部分列は 0 個



この区間の増加部分列は 6 個



この区間の増加部分列は 0 個

→ 合計 $1 + 0 + 6 + 0 = 7$ 通り

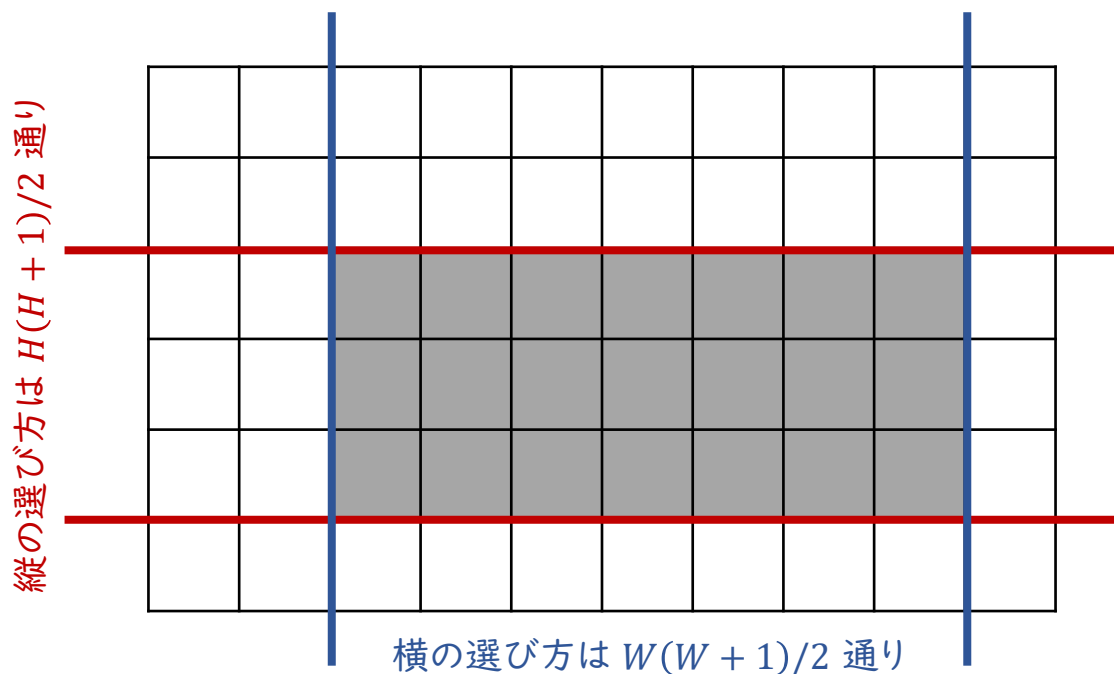
それぞれの区間に対し $L(L-1)/2$ を足すことで
答えが計算量 $O(W)$ で求められます



小課題 2 の解法

この小課題の制約は「 $HW \leq 100$ 」… 長方形領域を 全探索 したい

長方形領域の数は、大体 $H^2W^2/4$ 通り



小課題 2 の解法

各長方形領域に対して「全マス通れるか判定」したい

《判定のアルゴリズム》

1. マスを数が多い順にソートする
2. マスの数の大きい順に移動する時に、すべての移動が隣のマスになるのか判定

14	→	10	→	7	8	9
2	→	1		6	12	11
3	←	4	←	5	13	15

達成可能な例

14	10	7	8	9
2	1	6	12	11
3	4	5	13	15

達成不可能な例

小課題 2 の解法

これで、 $h \times w$ の長方形領域に対して、判定が計算量 $O(hw \log hw)$ でできます

よって、この問題は計算量 $O(H^3 W^3 \log HW)$ で解けました
小課題 2 に通ります



小課題 3 の前に

ここでクイズです!

Q すべての長方形領域のサイズの合計は、定数倍も含めてどのくらいになる?

A $O(HW)$ のサイズの長方形領域が $O(H^2W^2)$ 個なので、合計は $O(H^3W^3)$ 定数倍は…?

$$\frac{1}{4} \times \frac{1}{3} \times \frac{1}{3} = \frac{1}{36}$$

長方形領域の個数 縦の長さの期待値 横の長さの期待値

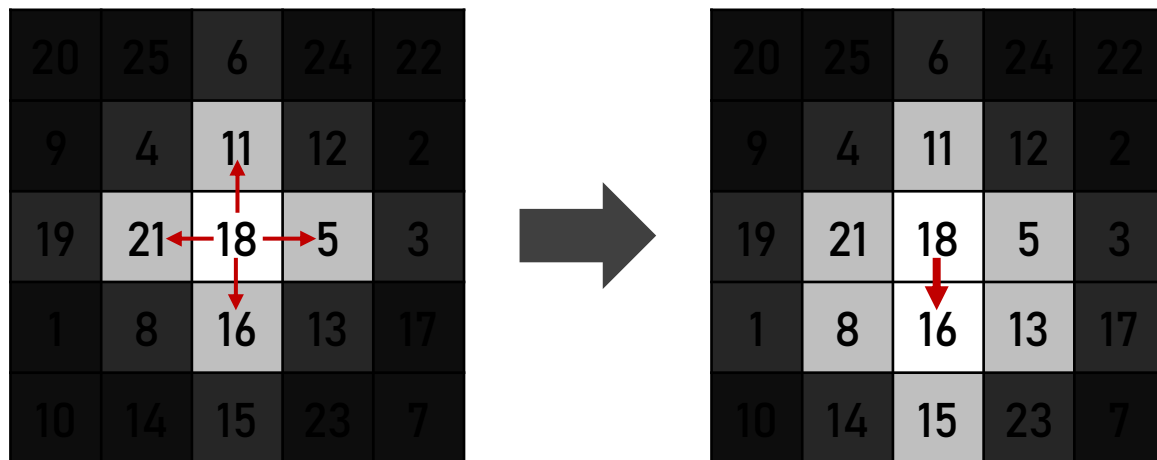
なので、計算量 $O(H^3W^3)$ なら $HW \leq 1500$ でも十分高速に動作しそう

小課題 3 の解法

あなたが JOI 君になりきって、砂の城の上を歩くことを考えます

Q あなたならどちら方向に進みますか？

A 下方向に進む（数が小さくなる中で最大のものに進むのが最適）



小課題 3 の解法

《判定のアルゴリズム》

1. 最も大きい数のマスからスタートする
2. 「数が小さくなる中で最大のものに進む」ことを繰り返したとき、長方形領域内のすべてのマスを訪れるか判定

各長方形領域に対し $O(hw)$ で判定できるので、全体計算量 $O(H^3W^3)$ で十分高速

14	→	10	→	7	8	9
2	→	1		6	12	11
3	←	4	←	5	13	15

達成可能な例

14	10	7	←	8	9	
2	←	1	→	6	12	11
3	4	←	5	13	15	

達成不可能な例



小課題 4 の目標

小課題 4 の制約は「 $HW \leq 7000$ 」

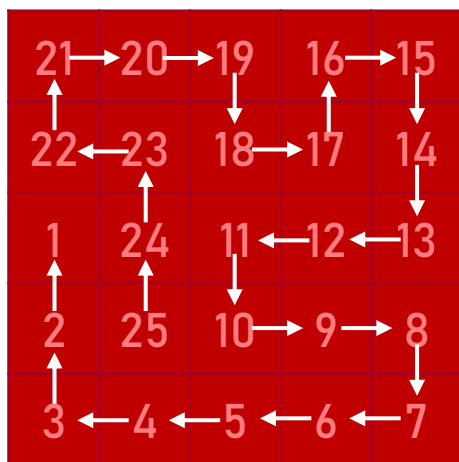
計算量 $O(H^2W^2)$ とかで解かなければならない

小課題 4 の目標

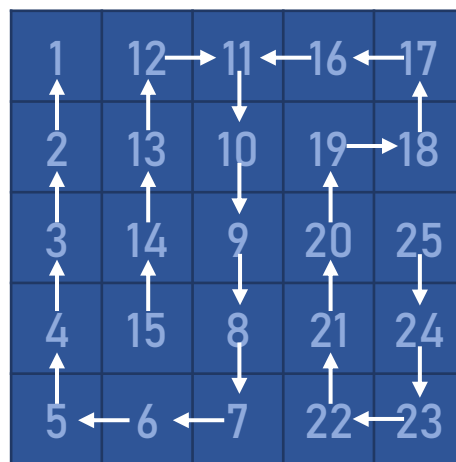
各長方形領域に対して
計算量 $O(1)$ で判定する

小課題 4 の解法のコネプト

各マスから出る矢印の方向は、最初から決まっている



↓ 1つのパスになっている
→ 達成可能



↓ 1つのパスになってない
→ 達成不可能

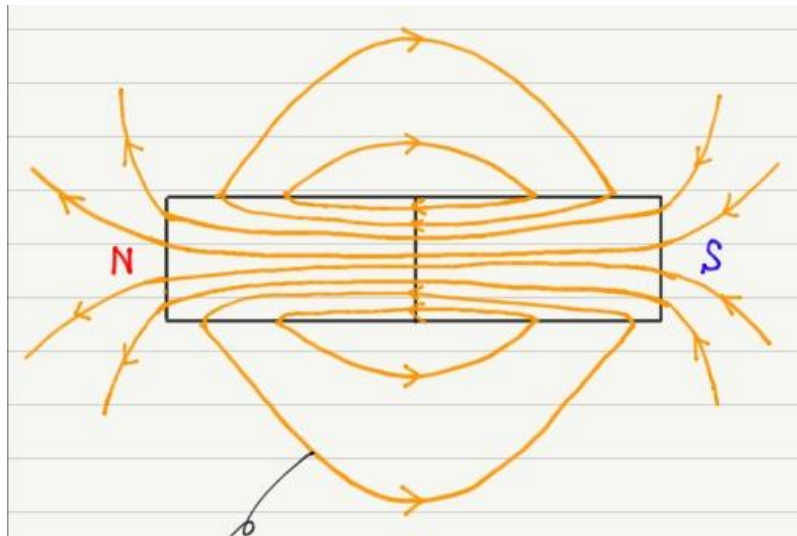
矢印が ↓ 1つの流れになっているかを、計算量 $O(1)$ で判定したい! どうやって?

小課題 4 の解法のコネプト

ところで、物理の法則について考えてみよう

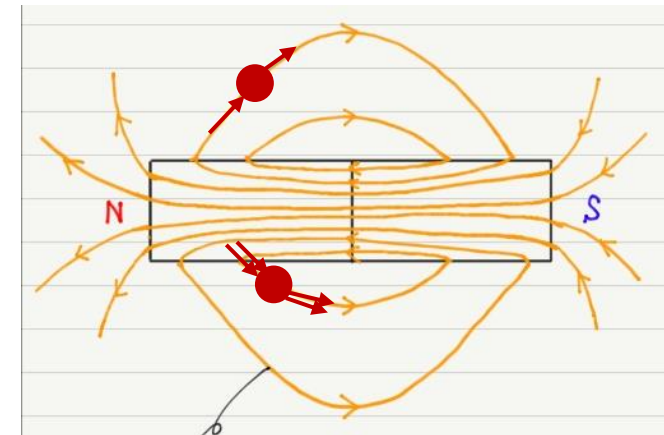
磁束線

磁束線は、1つのサイクルを描く



(画像は <https://www.yukimura-physics.com/entry/elemag21> より引用)

これは以下のように言い換えることができる
「すべての点において、入ってくる磁場と出ていく磁場が同じ」(磁場のガウスの法則)



小課題 4 の解法のコネプト

この問題にも同じようなアイデアが使えないか？

「1 つの流れになっている」

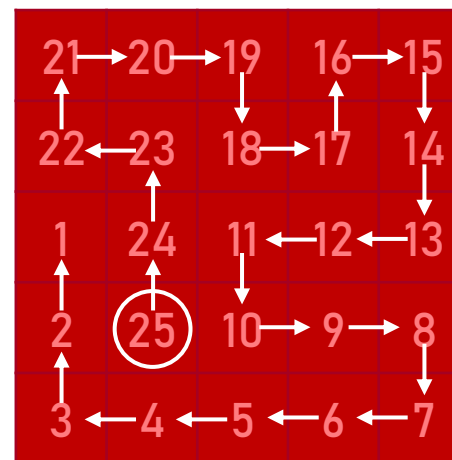


「入ってくる矢印の本数がすべて 1 本
(ただし始点は除く)」

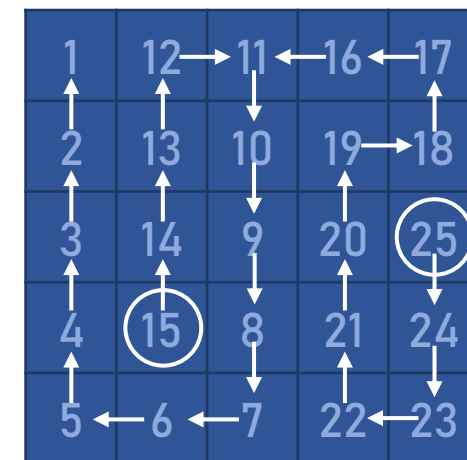
※ 出ていく矢印の本数は常に 1 本なので、流れをつなげる
ためには入ってくる矢印の本数は 1 本でなければならない



「入ってくる矢印が 0 本のマスが 1 個だけ」



1 個だけなので OK



2 個あるので NG

小課題 4 の解法

「入ってくる矢印が 0 本」のマスの個数を $O(1)$ で数えたい!

そもそも、入ってくる矢印は、長方形領域によってどう変わるか?

例：左から入る矢印の場合

15	21	20	19	16
14	22	23	18	17
2	1	24	11	12

左は領域の外なので
矢印は入ってこない

15	21	20	19	16
14	22	23	18	17
2	1	24	11	12

左から矢印が
入ってくる

15	21	20	19	16
14	22	23	18	17
2	1	24	11	12

左のマスからは外に矢印が出るので
矢印は入ってこない

15	21	20	19	16
14	22	23	18	17
2	1	24	11	12

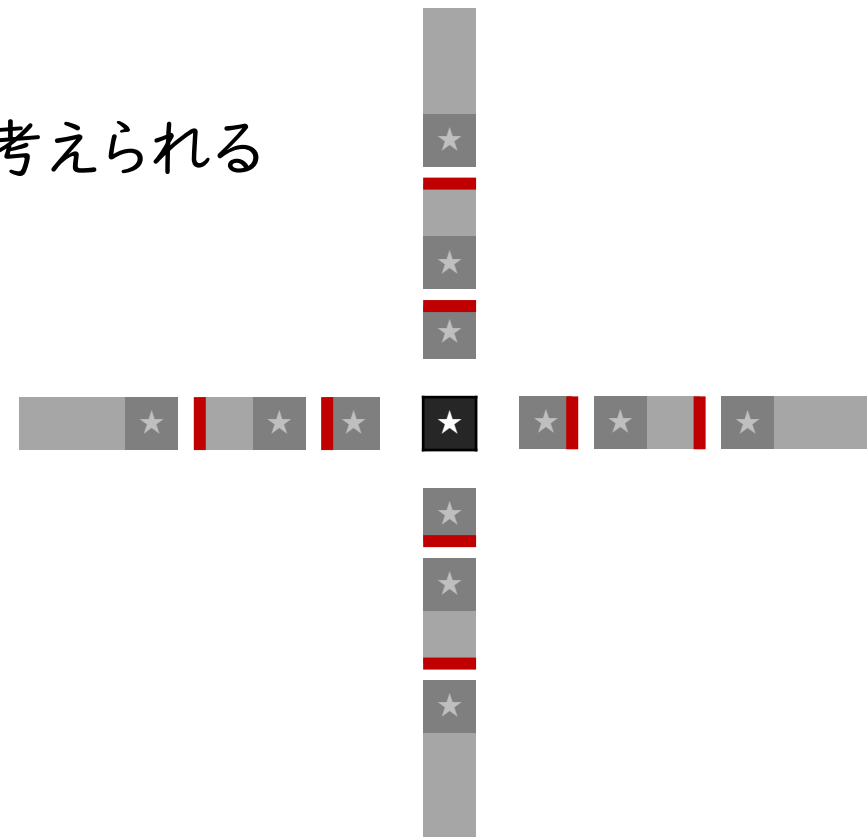
境界が 2 マス先以上なら
入り方は全部同じ

小課題 4 の解法

矢印の入りは、境界が「そのマスか」「1 マス先か」「2 マス以上先か」で変わる

上下左右 4 方向あるので、全部で $3^4 = 81$ パターン考えられる

- このすべてに対して、「入ってくる矢印が 0 本」のマスの個数を **2 次元累積和** で記録
- つまり、 $H \times W$ の 2 次元累積和を 81 個作る



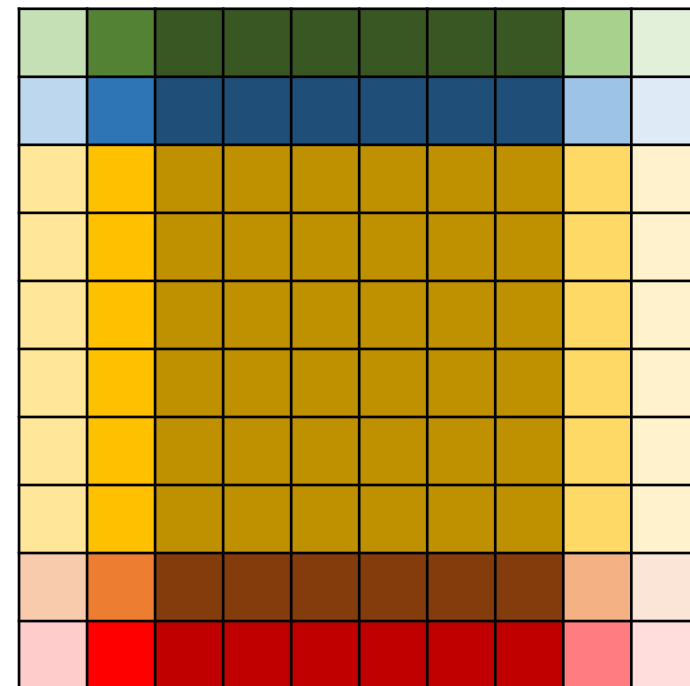
小課題 4 の解法

すると…「入ってくる矢印が 0 本」のマスが $O(1)$ で数えられる!

例えば、 $h \times w$ の長方形領域 ($h \geq 5, w \geq 5$) の場合

- 真ん中の $(h - 4) \times (w - 4)$ は、全方位「境界が 2 マス以上先」なので、同じ累積和で計算できる
- それ以外も含めて、右図の 25 通りに分けて計算できる

$h \leq 4$ または $w \leq 4$ の場合も、同様に計算できる



小課題 4 の解法

これで、 $h \times w$ の長方形領域に対して、判定が計算量 $O(1)$ でできます

よって、この問題は計算量 $O(H^2W^2)$ で解けました
小課題 4 に通ります

※ 実装によっては実行時間制限に間に合わないこともありますが、前スライドの 25 通り中 16 通りは「1 マスだけ」なので、これを 2 次元累積和でなく直接計算する工夫などで、十分に高速化できます。

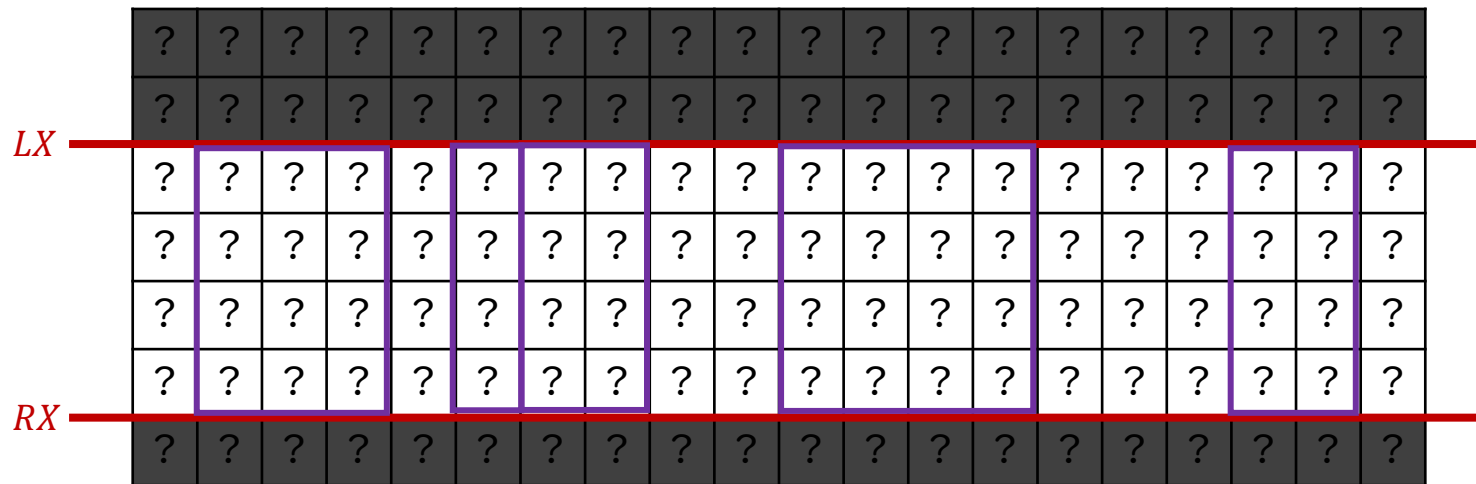


小課題 5 のアイデア

小課題 5 の制約は「 $HW \leq 50000$ 」

各長方形領域を $O(1)$ で判定することも許されない

アイデア：長方形領域の上と下を固定したときの数え上げを $O(W)$ でできないか？



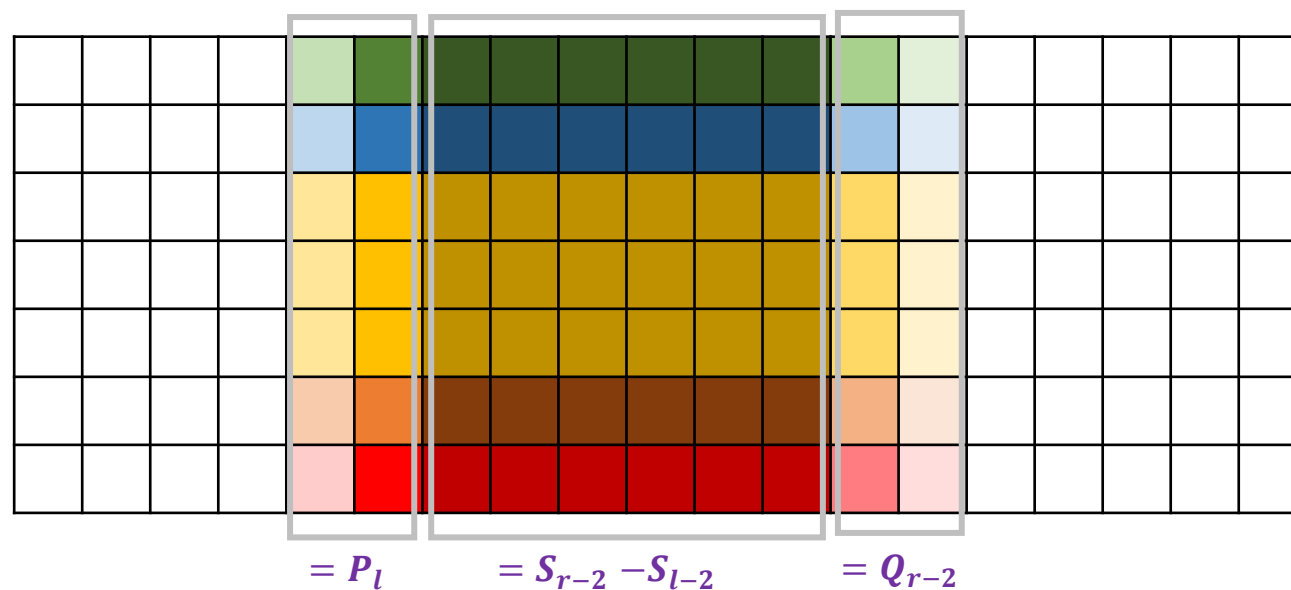
(LX, RX) を全探索

小課題 5 のアイデア

実は、ある区間の「入ってくる矢印が 0 本」のマス目の個数は、ほぼ累積和で表せる
(これをスコアとする)

$w \geq 5$ の場合、中央の長さ $w - 4$ の区間のスコアは $S_{r-2} - S_{l+2}$ と表せる

なので、区間のスコアは、 $B_r = S_{r-2} + Q_{r-2}$, $A_l = -S_{l+2} + Q_l$ として $B_r - A_l$ と表せる
(P_i, Q_i は下図参照)



小課題 5 の解法

したがって、次の問題を解けばよい

$B_j - A_i = 1$ となる (i, j) ($0 \leq i < j \leq W$) の組み合わせはいくつか？

$A_i, B_j \leq HW$ であるため、この問題は配列を使って $O(W)$ で解ける

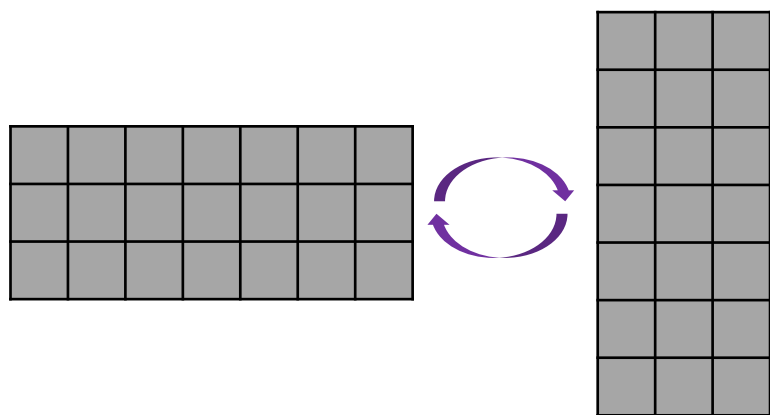
長方形の上端と下端を全探索するので、全体計算量 $O(H^2 \times W)$ で解ける

これだと $H \gg W$ の場合に時間がかかるが...

小課題 5 の解法

これだと $H \gg W$ の場合に時間がかかるが...

- 長方形を 90 度回転して同じ問題を解くと、 $O(H \times W^2)$ になる
- 両解法のうちよい方を採用すると、計算量 $O(HW \times \min(H, W))$ になるので...
- $\min(H, W) < \sqrt{HW}$ だから、計算量の中身は $(HW)^{1.5}$ になるので AC!!



統計情報

